

南京工业大学

2026 届毕业设计（论文）

题 目：	面向 F2FS 文件系统的 Large Folios 机制研究与实现
专 业：	人工智能
班 级：	智能 2202
姓 名：	赵南哲
指导老师：	万夕里
起讫日期：	2026 年 3 月 5 日至 2026 年 6 月 10 日

2026 年 5 月

南京工业大学本科毕业设计（论文）原创性声明

本人郑重声明：所呈交的毕业设计（论文），是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的作品成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律结果由本人承担。

论文作者签名： 日期： 年 月 日

南京工业大学本科毕业设计（论文）使用授权声明

本学位论文作者完全了解学校有关保留、使用毕业论文（设计）的规定，同意学校保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。本人授权南京工业大学教务处可以将本毕业论文（设计）的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编毕业论文（设计）。

论文作者签名： 导师签名：

日期： 年 月 日 日期： 年 月 日

面向 F2FS 文件系统的 Large Folios 机制研究与实现

摘 要

随着移动端本地数据规模和端侧人工智能模型文件体量持续增长，应用启动、资源装载、模型加载和后台更新等过程对文件系统的连续 I/O 能力提出了更高要求。

F2FS 作为 Android 设备中常用的闪存友好文件系统，具有顺序追加写入、垃圾回收和压缩文件等面向闪存的优化机制，但其原有缓冲 I/O 路径长期以 4KB 页和文件系统块为基本处理单位，难以充分利用底层块设备对大粒度连续 I/O 的支持。

可变阶数动态大页（Large Folio）能够扩大页缓存对象粒度，减少缓存对象数量和映射查询开销，而 F2FS 原有缓冲 I/O 路径仍围绕 4KB 页和文件系统块组织，难以直接承接一个页缓存对象覆盖多个文件系统块的情况。针对这一适配问题，本文将动态大页引入 F2FS 缓冲 I/O 路径，为 F2FS 支持大粒度连续文件 I/O 提供页缓存基础，并设计实现了一套基于 iomap 的动态大页缓冲 I/O 适配框架。该框架将 F2FS 的块级映射结果转换为面向文件字节区间的 iomap 描述，并扩展动态大页内部逐页状态对象，使动态大页能够在 F2FS 中完成区间化读写、子范围状态维护和 I/O 完成判断，并在此基础上对于 F2FS 专属的垃圾回收和压缩文件路径进行了针对性扩展，使后台块搬移和簇级压缩处理能够在兼容原有语义的同时支持动态大页，最终使 F2FS 能够在兼容顺序追加写入、垃圾回收和压缩文件等闪存友好机制的前提下，将动态大页应用到完整的缓冲 I/O 关键路径中。

本文在 QEMU 虚拟机和树莓派平台上对所实现的原型系统进行了实验评估。实验结果表明，相比未接入 iomap 动态大页路径的基线系统，本文框架在大块顺序 I/O 场景下能够显著提升 F2FS 的吞吐能力，其中 QEMU 平台上的普通文件写入带宽获得约 2-3 倍提升，树莓派平台上的普通文件、稀疏文件和压缩文件写入场景最高获得约 3-4 倍以上提升；同时，CPU 开销和带宽稳定性结果表明，该框架能够降低小粒度页缓存管理和逐块 I/O 组织带来的额外开销。实验结果验证了 F2FS 支持动态大页缓冲 I/O 的可行性和有效性。

关键词：F2FS；动态大页；页缓存；iomap；缓冲 I/O；压缩文件

Research and Implementation of the Large Folios Mechanism for the F2FS File System

ABSTRACT

As local data volume on mobile devices and the file size of on-device artificial intelligence models continue to grow, processes such as application startup, resource loading, model loading, and background updates impose higher requirements on the sequential I/O capability of file systems. F2FS, as a flash-friendly file system commonly used on Android devices, provides flash-oriented optimizations such as sequential append writing, garbage collection, and compressed files. However, its original buffered I/O path has long used 4 KB pages and file system blocks as the basic processing units, making it difficult to fully utilize the support of underlying block devices for large-granularity sequential I/O.

Variable-order large folios, namely Large Folios, can enlarge the granularity of page-cache objects and reduce the number of cache objects as well as mapping lookup overhead. However, the original F2FS buffered I/O path is still organized around 4 KB pages and file system blocks, making it difficult to directly handle the case where a single page-cache object covers multiple file system blocks. To address this adaptation problem, this thesis introduces large folios into the F2FS buffered I/O path, providing a page-cache foundation for F2FS to support large-granularity sequential file I/O. Based on this, this thesis designs and implements an iomap-based large-folio buffered I/O adaptation framework. The framework converts F2FS block-level mapping results into iomap descriptions based on file byte ranges, and extends the per-page state object inside large folios, enabling large folios in F2FS to support range-based read and write operations, subrange state maintenance, and I/O completion determination. On this basis, targeted extensions are further made for F2FS-specific garbage collection and compressed file paths, so that background block migration and cluster-level compression processing can support large folios while remaining compatible with their original semantics. As a result, F2FS can apply large folios to the complete key paths of buffered I/O while remaining compatible with its flash-friendly mechanisms, including sequential append writing, garbage collection, and compressed files.

This thesis evaluates the implemented prototype system on both QEMU virtual machines and a Raspberry Pi platform. Experimental results show that, compared with the baseline system without the iomap-based large-folio path, the proposed framework can significantly improve F2FS throughput under large-block sequential I/O workloads. Specifically, ordinary file write bandwidth on the QEMU platform is improved by

approximately 2-3 times, while ordinary file, sparse file, and compressed file write scenarios on the Raspberry Pi platform achieve up to more than 3-4 times improvement. Meanwhile, CPU overhead and bandwidth stability results show that the framework can reduce the extra overhead caused by fine-grained page-cache management and block-by-block I/O organization. These results demonstrate the feasibility and effectiveness of supporting large-folio buffered I/O in F2FS.

Key Words: F2FS; Large Folio; Page Cache; iomap; Buffered I/O; Compressed File

目 录

摘 要.....	3
ABSTRACT.....	4
第一章 绪论.....	9
1.1 研究背景与意义.....	9
1.2 国内外研究现状与挑战.....	12
1.2.1 页粒度与大页机制相关研究.....	12
1.2.2 文件系统映射与 iomap 相关研究.....	13
1.2.3 F2FS 与闪存友好文件系统相关研究.....	14
1.3 本文主要工作.....	14
1.4 论文组织结构.....	15
第二章 背景知识与关键技术.....	16
2.1 页缓存与页描述符.....	16
2.2 F2FS 块映射.....	17
2.3 块设备层 I/O 请求结构与多页提交能力.....	19
2.4 闪存逻辑地址空间与 FTL.....	20
2.5 读文件完整路径解析.....	21
2.6 可变阶数动态大页机制.....	22
2.7 F2FS 的异地更新、垃圾回收与压缩文件.....	23
2.8 iomap 框架的区间映射、路径迭代与动态大页内部状态.....	25
2.9 本章小结.....	28
第三章 F2FS 支持动态大页的关键问题分析.....	29
3.1 本章定位.....	29
3.2 F2FS 原 I/O 路径中的单页单块假设.....	29
3.3 F2FS 私有标志与 iomap 状态对象的字段冲突.....	30
3.4 动态大页跨越压缩簇与非压缩区间的问题.....	31
3.5 脏页写回中的脏区丢失与块索引错位.....	33

3.6 本章小结.....	33
第四章 F2FS 动态大页缓冲 I/O 设计与实现.....	34
4.1 总体架构.....	34
4.2 普通文件的区间映射 I/O.....	35
4.2.1 块级映射到字节区间的转换.....	35
4.2.2 普通文件缓冲读路径.....	37
4.2.3 普通文件缓冲写路径.....	38
4.3 F2FS 扩展型动态大页内部状态对象.....	40
4.4 压缩文件的动态大页读写适配.....	44
4.4.1 压缩文件缓冲读适配.....	44
4.4.2 压缩文件缓冲写适配.....	48
4.5 动态大页脏页写回.....	51
4.5.1 普通文件动态大页脏区间写回.....	51
4.5.2 压缩文件动态大页写回与延迟解锁.....	54
4.6 垃圾回收路径的动态大页适配.....	55
4.7 本章小结.....	56
第五章 实验设计与性能评估.....	57
5.1 测试环境与配置.....	57
5.2 测试方法与负载设计.....	58
5.2.1 工作负载.....	58
5.2.2 测试矩阵.....	58
5.2.3 评价指标.....	58
5.3 实验结果与分析.....	59
5.3.1 QEMU 虚拟机平台.....	59
5.3.2 树莓派 5 平台.....	63
5.3.3 CPU 利用率分析.....	73
结语.....	76

参考文献.....	77
致谢.....	79

第一章 绪论

1.1 研究背景与意义

近年来，移动终端上的应用形态正在发生明显变化。一方面，社交、办公、影像处理等应用持续增加本地数据规模，应用启动、资源装载和后台更新过程对文件系统吞吐能力提出了更高要求；另一方面，端侧人工智能模型逐步进入实际应用场景，大语言模型、多模态模型和视觉推理模型通常以较大文件形式存放在本地，其加载、缓存和增量更新过程都会产生持续的大文件顺序输入/输出（Input/Output, I/O）需求。与传统以小文件和零散访问为主的移动应用相比，这类负载更强调连续数据读取能力、页缓存（page cache）命中效率以及后台写回开销控制。文件系统如果仍长期围绕较小粒度组织页缓存和 I/O，往往难以充分发挥当前闪存设备的顺序访问能力^{[1][2]}。

在 Android 设备中，闪存友好文件系统（Flash-Friendly File System, F2FS）已成为常见的本地文件系统之一。F2FS 面向与非型闪存（Not AND, NAND）和基于闪存转换层（Flash Translation Layer, FTL）的块设备设计，强调顺序追加写入、冷热数据分离以及垃圾回收效率，对移动终端场景具有适配性^[3]。随着移动设备存储带宽和容量持续提升，F2FS 不仅承担普通应用文件访问，还直接影响模型文件装载、媒体资源处理和后台维护任务的整体开销。因此，围绕 F2FS 文件访问路径的缓存粒度和 I/O 组织方式展开研究，并提升现代应用场景下面的性能，具有实际优化价值。

在 Linux 内核中，页缓存通过一种称为页描述符（struct page）的数据结构管理每一个被缓存的数据单元^[4]。每一个页描述符对应 4KB 的物理内存，记录该内存页的引用计数、映射归属、回写状态等元数据。页缓存每缓存 4KB 文件数据，就需要分配和维护一个独立的页描述符，这意味着文件访问的缓存管理粒度在 4KB 基线配置下以 4KB 页为主。当应用需要读取一段较大的连续文件区间时，页缓存必须围每个 4KB 页描述符逐一建立状态、逐一查找映射、逐一组织 I/O 提交。在这个过程中，元数据维护、锁操作和映射查询的开销与需要处理的页描述符数量成正比。

而在页缓存之下是更底层的块层（block layer），它负责将读写请求组织为块设备 I/O（block I/O, bio）请求并提交到存储设备^[5]。尽管块层早已具备承载更大粒度 I/O 的能力，单个块设备 I/O 请求可以合并多个物理连续的块，闪存设备本身也倾向于接收较大的顺序访问请求，然而只要页缓存仍以 4KB 页描述符为对象组织缓存和状态，上层路径的调度和控制就始终受限于小粒度对象，块层和设备的连续 I/O 能力无法在上层得到充分利用。这就是目前 F2FS 为适应需要大块数据 I/O 负载所面对的第一层问题：4KB 页缓存粒度限制了对底层大块连续 I/O 能力的利用。

围绕上述问题，一个自然的思路是使用更大的内存页。Linux 内核提供透明大页（Transparent Huge Pages, THP）机制，在满足条件时自动将多个连续的 4KB 页合并为 2MB 的大页，以减少页描述符数量和地址转换开销^{[6][7]}。但 THP 主要面向匿名内存（anonymous memory，即进程的堆、栈等不与文件关联的私有内存），其对文件页缓存路径的适用性很有限。首先，2MB 的固定粒度对于文件访问而言过大，一次通常的文件读写在数十到数百 KB 量级，强制分配 2MB 会造成严重的内部碎片，例如应用实际只使用其中少量 4KB 子区域，却需要为整个 2MB 分配物理内存和维护页表映射。其次，维持 2MB 大页需要系统频繁执行内存压缩来寻找物理连续的 2MB 空间，在长期运行的移动设备上，物理内存碎片化使得这类高阶分配的成功率持续下降，系统在大页折叠与回退到 4KB 小页之间反复切换，反而引入额外的 CPU 开销和延迟抖动。正因为 THP 的粒度固定、面向场景单一且缺乏对文件访问模式的自适应能力，它并不能作为大型文件页缓存路径的直接解决方案。

为此，Linux 内核近年来逐步引入可变阶数动态大页(Large Folio,Linux 内核对大页的抽象描述符,以下简称"动态大页")机制^[8]。其在类型层面对页描述符做一个关键改进：一个动态大页必定对应一个完整的、可独立管理的缓存对象，而不会指向一个已聚合页组中的附属页。这一类型保证消除了内核代码中对页指针归属的反复判断，使页缓存可以自然地管理不同大小的缓存对象。当文件系统声明自身支持更大粒度的缓存对象后，页缓存可以在合适的访问模式下分配阶数（order）大于等于 1 的动态大页。阶数决定了动态大页的大小：阶数为 2 时对应 16KB（4 个连续 4KB 页），阶数为 4 时对应 64KB（16 页），依此类推。与 THP 不同，动态大页的阶数不是固定预设的，而是由页缓存在运行时根据文件访问的连续范围和内存空闲状况动态决定。因此，相比固定 2MB 粒度的 THP，动态大页更适合作为文件页缓存扩大粒度的基础。它既能够通过更大的缓存对象减少页描述符数量、锁操作次数和映射查询频率，又能够根据文件访问范围和内存空闲状况选择合适阶数，避免在中等规模文件 I/O 中引入过大的内存碎片和分配开销。

然而，第二个问题因此浮出水面,因为动态大页只是让页缓存对象变大，F2FS 的内部路径并不会因此自动适配。第一,一个动态大页覆盖的是连续文件字节区间，而 F2FS 原有块映射机制返回的是以逻辑块号为单位的物理映射结果。若继续沿用逐块查询和逐块提交方式，文件系统需要在同一个动态大页内反复处理多个块的映射关系，无法充分利用底层块层对连续 I/O 的合并能力。第二，一个动态大页内部可能包含多个 4KB 子块，这些子块可能分别处于未就绪、已就绪、已修改或正在写回状态，传统路径中“一个页缓存对象对应一个状态”的处理方式不再适用。第三,一个动态大页可能被拆分为多个块设备 I/O 请求提交，各请求的完成时间并不一致，传统路径中

“一次块设备 I/O 请求完成即一个页完成”的判断方式也不再适用。读完成、写回收尾以及动态大页状态更新，都需要能够感知动态大页内部不同子范围的 I/O 完成情况，而当前 F2FS 所有的状态更新和完成判断，还是只按照单个页描述符为一个单元判断，没有对具体的子范围 I/O 具体完成状态感知。

为了应对上述问题，Linux 内核提供了 iomap 框架^[9]。iomap 以文件字节区间为单位描述逻辑文件偏移到物理存储位置之间的映射关系，并以该区间作为缓冲 I/O、直接 I/O 和写回路径的统一组织基础。相比传统逐块映射方式，iomap 使文件系统能够围绕动态大页覆盖的文件范围返回尽可能大的连续映射区间，从而减少逐块查询和逐块提交带来的路径开销。同时，iomap 提供了按动态大页维护的逐块状态追踪和完成计数机制，使一个动态大页对应多个块设备 I/O 请求时，仍能够在所有相关 I/O 完成后判定读完成或写回结束。

但是，引入 iomap 以后，F2FS 又面临第三个问题：iomap 的通用状态管理方式与 F2FS 原有私有机制之间存在兼容冲突。F2FS 原有代码会在页缓存对象的私有存储空间中编码 垃圾回收 迁移状态、原子写标记等私有信息，而 iomap 框架也需要利用同一存储空间保存按动态大页维护的逐块状态位图。这样一来，同一个字段在两条代码路径中被赋予了不同语义：F2FS 将其作为私有标志使用，iomap 则将其作为状态结构指针使用。如果同一个动态大页先后经过 F2FS 私有代码路径和 iomap 代码路径，就可能出现字段所有权冲突，轻则造成状态误判，重则导致指针错误解引用甚至内核崩溃。

此外，F2FS 的 垃圾回收和压缩文件处理并不是普通缓冲 I/O（缓冲 I/O）代码路径的简单延伸。垃圾回收需要在后台读取、锁定并迁移有效块，压缩文件则以压缩簇为单位组织数据和状态。动态大页进入这些代码路径后，一个动态大页内部可能同时覆盖多个块、多个状态位，甚至与压缩簇边界发生交叉。如果 iomap 只处理普通读写代码路径，而没有同步覆盖 GC 和压缩文件代码路径，就会导致不同代码路径对同一动态大页的状态解释不一致，进而破坏 F2FS 原有的空间回收、压缩处理和原子写语义。因此，F2FS 支持动态大页不能只是简单接入 iomap，还必须在 iomap 的通用状态框架之上进行 F2FS 专属扩展。

基于上述背景，本文围绕 F2FS 支持动态大页的三层递进问题展开研究：第一层，利用动态大页扩大页缓存对象粒度，缓解 4 KB page 在大文件连续访问场景下带来的对象数量和管理开销问题；第二层，借助 iomap 的区间映射能力，将 F2FS 的 I/O 组织方式从逐块处理提升为面向文件字节区间的处理，使大粒度页缓存对象能够更好地匹配底层连续 I/O 提交；第三层，通过扩展 iomap 的统一状态结构和 F2FS 专属适配，解决私有字段兼容、GC 路径协同和压缩文件支持等 F2FS 特有的工程问题。

上述研究的意义主要体现在三个方面。第一，在应用负载层面，移动端大模型、影像处理和媒体资源管理等场景正在增加大文件顺序访问需求，优化 F2FS 的页缓存和 I/O 组织方式，有助于降低模型文件加载、资源装载和后台维护过程中的路径开销。第二，在系统软件层面，Android 生态正在从长期使用 4 KB 页面大小逐步走向支持 16 KB 页面大小的阶段^[10]，这一趋势表明更大内存管理粒度已成为移动系统优化方向之一，本研究与移动系统向更大内存管理粒度和更高效 I/O 路径演进的趋势相契合。第三，在文件系统实现层面，F2FS 作为 Android 设备中广泛使用的闪存友好文件系统，其动态大页支持不能只停留在页缓存层，还必须同步处理映射、写回、GC、压缩和私有状态管理等完整路径。因此，围绕 iomap 扩展 F2FS 的动态大页支持，不仅具有提升顺序 I/O 性能的工程价值，也为后续移动端文件系统适配更大页面粒度和更复杂 I/O 负载提供了实现参考。

1.2 国内外研究现状与挑战

1.2.1 页粒度与大页机制相关研究

围绕内存页粒度的研究，研究人员长期关注的核心问题在于小页的灵活性与大页的映射效率之间如何平衡。早期研究指出，4KB 页面虽然有利于控制外部碎片，但在较大工作集和多级页表环境下，会缩小 TLB 覆盖范围并增加地址转换开销^{[11][12]}。随着内存容量增长和访问范围扩大，围绕大页、超页（superpage）以及地址转换成本的研究不断增多，相关工作普遍认为，在连续访问占比较高的场景中，扩大基本管理粒度有助于降低页表项数量和相关元数据维护开销^{[13][14]}。

在 Linux 系统中，透明大页曾被视为缓解小页管理成本的重要手段^[7]。然而，透明大页主要围绕匿名内存展开，固定 2MB 粒度在长期运行系统中容易受到内存碎片、高阶页分配困难以及内部碎片等问题影响。对于文件页缓存路径而言，透明大页的粒度过大且控制方式有限，难以稳定适应复杂文件访问模式。近年来，内核社区逐步推动动态大页机制取代传统以页描述符（struct page）为核心的部分接口，试图以更明确的类型表达单页或复合页整体，并为页缓存引入更灵活的大粒度缓存对象^[8]。这为面向文件系统的动态大页支持奠定了基础。

总体来看，已有研究已经从体系结构和内核实现两个层面说明了传统 4KB 页面在大范围连续访问场景中的局限。但现有大页研究仍主要关注内存管理侧的地址转换和碎片控制，未深入探讨页缓存粒度变化对文件系统 I/O 路径组织方式的影响。同时，块层大粒度块设备 I/O 请求和顺序 I/O 的研究聚焦于设备侧请求合并与调度策略，未向上延伸到页缓存对象粒度与文件系统映射语义的协同层面。页缓存层的粒度演进与文件系统层的 I/O 组织之间，仍缺少系统化的衔接研究。

1.2.2 文件系统映射与 iomap 相关研究

在传统 Linux 文件系统实现中，缓冲描述符（`buffer_head`）长期用于描述页内文件系统块与底层设备块之间的关系。该模型在页面（`page`）与块（`block`）粒度一致的情况下具有通用性，但随着页缓存对象和 I/O 请求粒度增大，其局限逐渐暴露。一方面，`buffer_head` 强依赖“页内多个小块状态对象”的组织方式；另一方面，路径上的读写、写回和完成处理也往往建立在逐块遍历基础上。当文件系统需要返回更大范围的连续映射时，这种面向单块对象的模型会增加状态维护和路径组织成本。

为解决这一问题，`iomap` 框架提出了区间映射思想^[9]。与传统按块返回结果的接口不同，`iomap` 直接以文件逻辑字节区间为单位描述底层映射关系，并通过统一的迭代器与回调接口支持 缓冲 I/O、直接 I/O（`direct I/O`）和 脏页写回（`dirty page writeback`）等路径。XFS 是较早全面采用 `iomap` 的文件系统，`ext4`、`ext2` 等文件系统也逐步围绕 直接 I/O 或 缓冲 I/O 迁移到 `iomap` 框架^[15]。与此同时，`Btrfs` 等现代 Linux 文件系统也体现出以区间、`extent` 和写时复制为核心的文件系统组织趋势^[16]。已有实践表明，区间化映射方式能够减少对每块状态对象的依赖，更适合与动态大页、大块顺序 I/O 和大块设备访问能力相结合^[17]。

现有研究已经说明了 `iomap` 的抽象优势和在主流文件系统推广趋势，但面向 F2FS 的协同适配仍处于持续演进阶段。具体而言，如何在 F2FS 的块级映射与 `iomap` 字节区间语义之间建立稳定的转换路径，如何在 `iomap` 逐动态大页维护的状态框架中容纳 F2FS 的 GC 和原子写等私有语义，以及如何使写回和压缩路径的完成判断适配 `iomap` 的原子计数机制，这三个层面的工程问题仍缺乏系统化的解决分析。

1.2.3 F2FS 与闪存友好文件系统相关研究

F2FS 最初作为面向 NAND 闪存设计的日志结构文件系统提出，其设计目标是在块设备抽象之上尽量保持顺序追加写入，降低随机覆盖更新带来的写放大，并借助冷热数据分离、多日志和垃圾回收机制改善闪存存储效率^{[3][18]}。相关研究和官方文档对 F2FS 的节点地址表（Node Address Table, NAT）、段信息表（Segment Information Table, SIT）、段摘要区（Segment Summary Area, SSA）、检查点（Checkpoint）以及段（Segment）、节（Section）、区（Zone）等空间组织方式进行了较系统说明，也讨论了其在移动设备和通用闪存场景中的应用价值^{[3][19]}。

除基础布局外，已有研究还关注了 F2FS 垃圾回收、压缩文件、冷热数据分离和块分配策略等问题。这些机制提升了 F2FS 对闪存设备的适应能力，但也使其文件访问路径较普通文件系统更复杂。例如，异地更新意味着写回并非原地覆盖，垃圾回收

需要读取并搬移有效块，压缩文件则引入了簇级处理语义^[20]。这些特性决定了 F2FS 在面对更大粒度页缓存对象时，不能简单沿用传统单页或单块路径。

总体而言，已有工作已经分别说明了大页机制、iomap 区间映射框架和 F2FS 闪存优化设计的必要性，但对本文所关注的三层递进问题尚未形成完整回答：现有研究证明了 4KB 粒度在连续访问场景中的局限，也为动态大页提供了基础；iomap 框架展示了区间化 I/O 组织的通用优势；但针对 F2FS 场景下如何将动态大页、iomap 与 F2FS 自身的 GC、压缩和私有状态机制协同适配，尚缺乏系统化的实现分析。

因此，本文面临的核心挑战并不是单独使用动态大页或单独接入 iomap，而是如何在 F2FS 的普通读写、脏页写回、垃圾回收和压缩文件路径中，同时解决缓存粒度扩大、区间映射转换和私有状态兼容三个问题。

1.3 本文主要工作

围绕上述研究背景与挑战，本文的主要工作如下：

第一，系统分析 F2FS 支持动态大页过程中涉及的三层递进问题：4KB 粒度过小对连续 I/O 能力的限制、逐块 I/O 组织与字节区间页缓存对象之间的不匹配、以及 F2FS 私有语义与 iomap 通用框架之间的兼容冲突。本文从这些问题出发，归纳了它们在普通文件读写、压缩文件处理、脏页写回和 GC 搬运场景中的具体表现，为后续方案设计提供依据。

第二，设计基于 iomap 的普通文件区间映射机制。针对 F2FS 原有块级映射结果与 iomap 字节区间语义之间的不一致，本文构建从逻辑块映射到文件字节区间映射的转换路径，使 F2FS 能够围绕动态大页覆盖范围返回稳定的连续映射结果。在此基础上，普通文件的缓冲读写能够在更大粒度页缓存对象上组织 I/O 提交，从而为后续回写控制和状态管理奠定基础。

第三，完成面向动态大页的状态管理与写回控制适配。针对一个动态大页内部可能包含多个文件系统块、并可能对应多个块设备 I/O 请求的情况，本文围绕动态大页私有状态组织、读完成判断、脏页子范围定位和子范围写回提交等关键环节，设计统一状态结构与控制流程，减少局部修改引发的扩大写回，改善大粒度页缓存对象在写回路径中的可控性。

第四，扩展 F2FS 在垃圾回收和压缩文件场景下的动态大页支持。针对 F2FS 的异地更新、有效块搬移和压缩簇处理机制，本文进一步将统一状态结构和 iomap 适配逻辑延伸到 GC 与压缩文件路径中，使普通缓冲路径中的优化不会与后台维护路径相互割裂。通过处理垃圾回收过程中的状态继承、压缩簇边界与动态大页子范围之间的关系，保证 F2FS 专属机制在动态大页条件下仍能保持一致的状态语义和正确的完成判断。

第五，通过实验评估验证所提出方案的有效性。本文在 QEMU 虚拟机和树莓派平台上构建测试环境，围绕顺序读写场景对比改造前后 F2FS 的性能表现，分析本文方案在大块 I/O 负载下对吞吐能力和路径开销的影响。

1.4 论文组织结构

第一章为绪论，主要介绍本文的研究背景、国内外研究现状与挑战，并概述本文的主要工作和全文结构。

第二章为背景知识与关键技术，先从一次 缓冲 I/O 系统调用出发说明数据从用户态到闪存的完整路径和层级关系，在页缓存层自然引出动态大页，随后介绍 F2FS 的日志结构、异地更新、GC 和压缩文件机制，最后说明 iomap 的区间映射思想及其与动态大页的协同关系。

第三章分析 F2FS 支持动态大页的关键问题，从映射边界不一致、状态语义冲突和 I/O 完成生命周期不一致三个方面，讨论动态大页进入 F2FS 后在不同路径中的具体困难。

第四章介绍 F2FS 动态大页缓冲 I/O 的设计与实现，重点说明普通文件区间映射、统一状态管理、脏页写回控制、GC 路径适配以及压缩文件场景扩展的具体方案。

第五章给出实验设计与性能评估结果，说明测试环境、工作负载、评价指标与结果分析，并验证本文方案在顺序 I/O 场景下的有效性。

第六章对全文工作进行总结，并展望后续可继续完善的研究方向。

第二章 背景知识与关键技术

2.1 页缓存与页描述符

在 Linux 内核中，所有对文件的缓冲读写都经过页缓存。页缓存位于虚拟文件系统层之下、具体文件系统之上，将磁盘上的文件数据缓存在内存中，以减少对存储设备的直接访问。

页缓存以页描述符（struct page）为基本管理单位。每个页描述符对应 4KB 物理内存，代表文件数据在内存中的一个缓存单元。struct page 有三个与本文直接相关的核心字段：index 记录该页在文件内的页索引——对于文件偏移 offset 处的数据，页索引的计算方式为 $\text{offset} / 4096$ （文件偏移除以页大小向下取整）；mapping 指向该页所属文件的地址空间（address_space），将页与文件关联；flags 通过一组状态位标记该页的当前状况——PG_uptodate 表示页中数据与磁盘一致或更新，PG_dirty 表示数据已被修改但尚未写回，PG_locked 表示该页正在被某条 I/O 路径操作，其他路径不得同时访问。可以把地址空间理解为某个文件在页缓存中的管理入口——页缓存通过它把文件偏移和内存页对应起来。表 2-1 汇总了这四种标志的含义。

表 2-1 页缓存对象级状态标志

状态标志	含义
PG_locked	对象正在被某条 I/O 路径操作
PG_uptodate	内存数据已就绪，可被读取
PG_dirty	数据已被修改但尚未写回
PG_writeback	对象正在写回磁盘

当应用通过 `read()` 系统调用请求读取一段文件数据时，内核根据文件偏移计算出页索引，在 索引节点 的 地址映射 中查找对应页描述符。如果存在且标记为 `uptodate`，数据直接从页缓存拷贝到用户缓冲区。如果不存在，则触发缺页流程：内核分配新的页描述符，对其加锁（设置 `PG_locked`），然后请求下层文件系统提供该页对应的磁盘数据位置。文件系统完成映射后，内核组织 I/O 请求从磁盘读取数据，数据到达后将页标记为 `uptodate` 并解锁。页缓存中的数据就绪后，系统调用将数据拷贝到用户缓冲区后返回。

上述流程中的全部操作——分配、加锁、标记状态、查找映射——均以单个 4KB 页描述符为粒度。一次 64KB 的读请求意味着十六次分配、十六次加锁、十六次映射查询。页缓存的管理粒度直接决定了 I/O 路径上元数据开销的放大倍数。

2.2 F2FS 块映射

页缓存知道需要读取哪个文件偏移范围的数据，但不知道这些数据在磁盘上的物理位置。这个信息由文件系统层提供。F2FS 使用基于节点（`node`）的索引结构来维护文件逻辑位置到物理存储位置的映射关系。

每个文件的 索引节点 块中存储了指向数据块的直接指针和指向间接节点块的指针。间接节点块再指向更下一层的数据块或节点块，形成一个多级索引树。

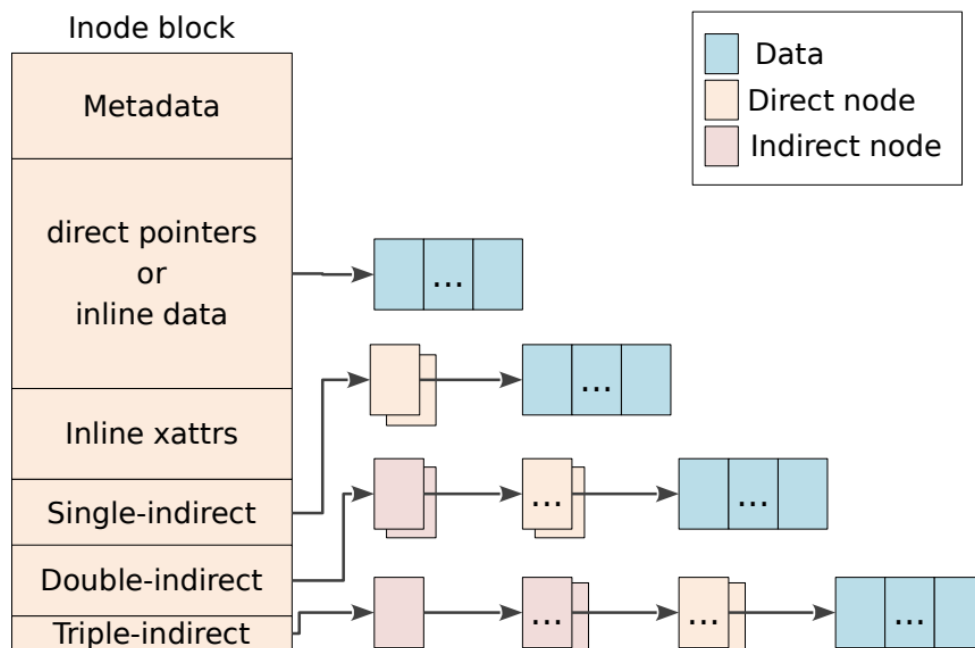


图 2-1 F2FS 索引节点与数据块映射结构

F2FS 通过节点地址表（Node Address Table，NAT）管理所有节点块的物理位置——NAT 维护从节点 ID 到节点块物理地址的映射，使文件系统能够在需要时快速定位并读取节点块。

当页缓存请求文件系统为某个文件偏移范围提供映射时，F2FS 的块映射入口函数（`f2fs_map_blocks`）执行以下流程：将文件偏移转换为文件内部逻辑块号（偏移量除以块大小，F2FS 块大小通常为 4KB）；从索引节点开始沿节点树逐级查找，直到定位到该逻辑块对应的直接节点条目。直接节点条目可以理解为 F2FS 节点树中最终记录某个逻辑块地址的位置——F2FS 找到该条目后，才能判断该逻辑块对应有效物理块、空洞、新分配块还是压缩簇。从该条目中读取物理块号，并统计该条目之后连续的、物理地址也连续的块数量。映射结果以逻辑块号、物理块号、连续块长度和区间标志（普通映射 `MAPPED`、空洞 `HOLE`、新分配 `NEW_ADDR` 等）的形式返回给页缓存层。

在传统 4KB 路径中，一次 F2FS 块映射函数调用通常返回与一个页描述符对应的一个物理块映射结果。页缓存拿到物理块号后，将映射结果交给块层，组织实际的磁盘读取请求。

F2FS 块映射函数沿节点树定位到目标逻辑块对应的直接节点条目后，条目中的块地址字段决定了该逻辑块的映射状态。以下四种状态覆盖了 F2FS 文件数据路径中所有可能的映射情况。

有效物理块地址是节点条目中存放了一个正常的物理块号，表示该逻辑块的数据已经写入磁盘的某个确定位置。这是在文件已有数据且已落盘后的常规状态。页缓存拿到物理块号后，将其换算为设备扇区地址即可构建块设备 I/O 请求发起磁盘 I/O。

NULL_ADDR 表示节点条目中该逻辑块对应的地址为零，该逻辑块当前没有物理映射。F2FS 内部进一步区分两种空洞：如果该逻辑块对应的直接节点条目根本不存在，说明这个文件偏移范围内连节点结构都未分配，空洞覆盖的范围以"下一个存在节点块的逻辑页号"为界；如果节点条目存在但地址为零，空洞范围以本次查询的连续块数为界。NULL_ADDR 常见于稀疏文件（lseek 跳过大段偏移后写入）和文件首次访问但尚未分配空间的位置。

NEW_ADDR 是 F2FS 内部的一个特殊标记，表示该逻辑块已被预留或新分配，但数据尚未真正写入磁盘。它典型地出现在两个场景中：延迟分配——write_begin 阶段通过 F2FS 预留新块函数（f2fs_reserve_new_block）为即将写入的数据预留了逻辑块位置，此时数据还在页缓存中未落盘；fallocate 预分配——系统调用提前为文件预留了连续的逻辑块空间。NEW_ADDR 不是可读的物理地址——如果读路径看到它，说明有预分配但无实际数据，需要填零处理；写回路径看到它，则说明需要为该块分配真正的物理地址后才能提交块设备 I/O 请求。

COMPRESS_ADDR 表示该逻辑块属于压缩簇，不能按普通物理块号解释。F2FS 的压缩机制以簇为单位将多个连续逻辑块压缩为一个或少数几个物理块写入磁盘，簇中第一个逻辑块在节点树中的地址被标记为 COMPRESS_ADDR，表明这里的物理布局需要按压缩语义解析（读取整个压缩段、解压后填充簇内全部逻辑块）。

下表汇总了这四种块映射状态。

表 2-2 F2FS 块映射状态

块映射状态	含义	典型产生场景
有效物理块地址	数据已落盘，有确定物理位置	已写入且 checkpoint 后的常规数据块
NULL_ADDR	无物理映射（空洞）	稀疏文件空洞、从未写入的位置
NEW_ADDR	已预留但数据未落盘	延迟分配、fallocate 预分配
COMPRESS_ADDR	属于压缩簇，需按压缩语义解析	压缩文件的数据路径

2.3 块设备层 I/O 请求结构与多页提交能力

映射结果到手后，内核需要向存储设备发出实际的读取命令。在 Linux 中，所有块设备的 I/O 请求均以 bio（block I/O request）的形式组织和提交。

bio 的核心数据结构包括两部分。第一部分是 `bi_iter`，记录本次 I/O 请求的目标区域：`bi_sector` 表示目标设备上的起始扇区号（以 512 字节为单位），由物理块号乘以 8 换算（一个 4KB 块等于 8 个 512 字节扇区）；`bi_size` 表示本次 I/O 的总字节数。第二部分是 `bi_io_vec`，这是一个 `bio_vec` 数组。每个 `bio_vec` 描述一段物理连续的内存区间，包含起始页指针 `bv_page`、页内偏移 `bv_offset` 和区间长度 `bv_len` 三个字段。一个 bio 最多可包含 256 个 `bio_vec` (`BIO_MAX_VECS`)。

bio 的数据结构本身可以描述多页 I/O，这种能力体现在两个层面。首先，多个物理地址不连续的页可以通过不同的 `bio_vec` 被同一个 bio 引用，合并为一次提交 bio 函数 (`submit_bio`) 提交。其次，`bio_vec` 自支持多页 `bvec` 以来，其 `bv_len` 不再限于一个页大小——早期一个 `bio_vec` 通常只覆盖一个页内范围，而多页 `bvec` 允许一个 `bio_vec` 用更长的 `bv_len` 覆盖连续多个页的物理内存——当一组物理连续页被加入同一个 `bvec` 时，`bv_len` 直接扩展为多页总长度，不拆分为多个单页 `bvec`。bio 添加页函数 (`bio_add_page`) 会检查新页的物理起始地址是否与已有 `bvec` 的末地址连续：若连续，则直接扩展该 `bvec` 的 `bv_len`；若不连续或 bio 已满，则新增一个 `bio_vec`。

这意味着，一个 64KB 的物理连续内存区间可以被一个 bio 中的少量 多页扩展 `bvec` 完整覆盖，`bvec` 数组的遍历开销相对于单页路径大幅降低。

bio 构建完成后通过 提交 bio 函数 提交到块设备队列。块设备驱动通过 DMA 将数据从存储设备传输到 bio 所描述的内存区域。

I/O 的完成通知是通过回调机制异步传递的，理解这一点对后续全文至关重要。当块设备完成一次 DMA 传输后，它并不直接返回给调用者，而是在中断上下文（或软中断 `bottom half`）中触发块设备 I/O 请求上预先注册的完成回调函数。这个回调在中断上下文中执行——它不会去获取页缓存层的任何锁，也不知道当初发起这个块设备 I/O 请求的上层代码当前处于什么状态。因此从上层视角看，块设备 I/O 请求的完成是一个异步事件：提交请求的代码已经返回了，页缓存对象的状态更新要等到未来的某个时刻由完成回调来写入。

对于读操作，完成回调的职责是将块设备 I/O 请求覆盖范围内的每一个页描述符的 `PG_locked` 标志清除，设置 `PG_uptodate`（表示内存中的数据已与磁盘同步），然后解锁该页。从这一刻起，上层代码可以安全地读取这些页中的数据。对于写操作，完成回调的职责是清除页描述符上的 `PG_writeback` 标志——表示该页的脏数据已安全写入磁盘，写回流程可以宣告结束，该页后续可以重新被标记为脏进入下一轮写回，或在内存紧张时被回收。

这一机制在动态大页条件下有一个直接后果：一个块设备 I/O 请求的完成回调只知道"我覆盖了哪些页"，但它不知道这些页是否属于一个更大的动态大页，也不知道同一个动态大页是否还有其他块设备 I/O 请求尚未完成。在传统 4KB 路径中，一个页就是一个独立对象，一个块设备 I/O 请求覆盖一个页，一次回调即为一次完整的生命周期事件——这不构成问题。但当页缓存对象变成动态大页后，一个动态大页的多个子范围可能被拆分到多个块设备 I/O 请求中，各块设备 I/O 请求的完成时机互不感知。于是，"整个动态大页的读是否完成"和"整个动态大页的写回是否结束"不再能从单个回调中判定——必须有一个跨回调的原子计数机制来汇总所有相关块设备 I/O 请求的完成状态。这正是第三章多处分析的问题起点，也是第四章统一状态结构中 未完成读字节计数 和 未完成写回字节计数 计数器的设计动机。

2.4 闪存逻辑地址空间与 FTL

块设备 I/O 请求提交到块设备队列之后，最终到达的是闪存存储设备。然而，块设备驱动程序所看到的连续逻辑地址空间，与闪存芯片内部的实际物理布局之间存在根本差异。这种差异由闪存转换层（Flash Translation Layer, FTL）在固件层面隐藏和管理。

NAND 闪存具有一个核心的不对称性：写入以页为单位（通常 4KB 至 16KB），但擦除必须以块为单位（通常包含 64 至 256 个页，即数百 KB 到数 MB）。一个闪存页在被擦除之前只能写入一次。若要原地修改一个页中的已有数据，必须先擦除整个包含它的擦除块——而擦除操作的延迟远高于写入（毫秒级对比百微秒级），且每个擦除块的可擦写次数有限。

FTL 的角色是向上层呈现一个普通的线性逻辑块设备，同时在内部维护从逻辑块地址（LBA）到物理闪存页的动态映射。当文件系统写入一个 LBA 时，FTL 将其映射到一个已擦除的空闲物理页，然后将原物理页标记为无效。FTL 在后台执行垃圾回收——将包含有效数据的页搬移到新的擦除块，然后擦除旧的擦除块以回收空间。整个过程对上层文件系统透明。

这种架构导致一个直接影响：如果上层文件系统频繁进行随机的小块原地覆盖写入，FTL 内部的垃圾回收压力会急剧增大，产生显著的写放大——实际写入闪存的数据量远超文件系统发出的写入量。相反，如果文件系统倾向于将写入组织为连续的大块追加，则 FTL 可以将这些写入映射到连续的物理区域，减少碎片和 GC 开销。这一认识直接导向下一节所讨论的 F2FS 设计选择。

2.5 读文件完整路径解析

现在用前四节的背景知识串联一个具体例子来展示完整的读路径。假设应用调用 `read(fd, buf, 65536)` 即请求从文件描述符 `fd` 对应的文件的偏移 0 开始读取 64KB 数据。

第一步：系统调用经 VFS 进入通用读路径 通用文件读迭代器 (`generic_file_read_iter`)，该函数循环处理 64KB 的读范围。对于偏移 0 处的数据，计算页索引为 0 ($0 / 4096 = 0$)，在文件 索引节点 的地址映射 中查找页索引为 0 的页描述符。假设页缓存为空（冷读），未命中。

第二步：页缓存分配一个新的页描述符 (`index=0`)，对其加锁 (`PG_locked`)。然后触发预读 (`readahead`)——预读窗口检测到顺序访问模式后，可能将这个 4KB 的单页请求扩展为更大的预读范围，一次性分配 `index=0` 到 `index=15` 共十六个页描述符。每个页描述符都被加锁，准备接收数据。

第三步：页缓存调用 F2FS 的 F2FS 预读函数 (`f2fs_readahead`) 获取映射。F2FS 内部将文件偏移 0 转换为逻辑块号 0，沿着节点树查找到对应的直接节点条目，得到物理块号（假设为 `P`）。继续检查后续块：逻辑块 1 映射到 `P+1`，逻辑块 2 映射到 `P+2`.....假设在本次调用中 F2FS 返回了连续 16 个块的映射结果 (`P` 到 `P+15`)。

第四步：块层接到映射结果后建立块设备 I/O 请求。将物理块号 `P` 换算为起始扇区号 `bi_sector = P * 8`。然后逐页调用添加页函数：对于 `index=0` 的页，加入其对应的物理内存段；对于 `index=1` 的页，因其物理地址与 `index=0` 的页连续（内核分配物理连续页面），被合并到同一个 `bio_vec` 中。十六个页全部加入后，该请求的总字节数为 65536。提交函数将块设备 I/O 请求提交到块设备队列。

第五步：块设备驱动通过 DMA 将 64KB 数据从闪存读入块设备 I/O 请求描述的内存区域。块设备 I/O 请求完成后，完成回调函数被触发，遍历块设备 I/O 请求所覆盖的每一个页描述符，将其 `PG_locked` 清除并设置 `PG_uptodate`，表示数据已就位。十六个页全部就绪。

第六步：`read()` 系统调用将页缓存中的数据逐页拷贝到用户缓冲区 `buf`。所有数据拷贝完成后，系统调用返回 65536。

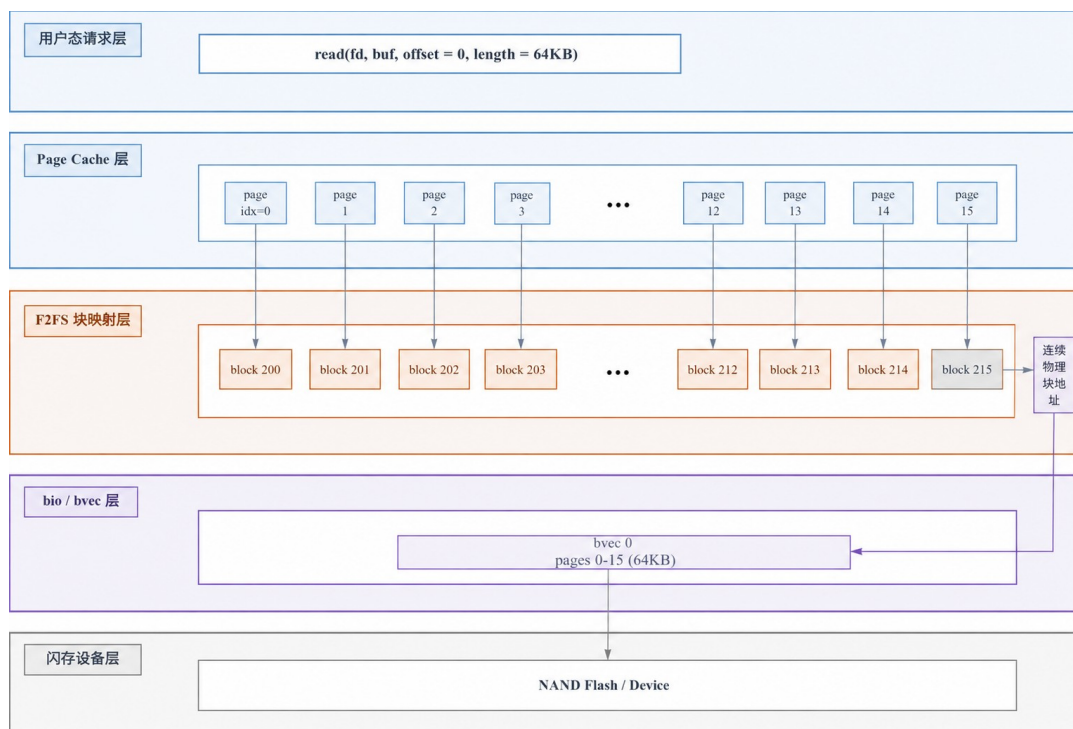


图 2-2 一次 64KB 缓冲读的完整路径

该流程表明：块层的块设备 I/O 请求有能力将十六个页的数据打包为一个块设备 I/O 请求提交——物理连续页还可以被合并到少量多页 `bvec` 中，进一步降低 `bvec` 数组的遍历开销——但页缓存层和文件系统映射层仍然围绕每个 4KB 页描述符独立操作，分配、加锁、映射查询各发生十六次。块层的聚合能力被上层细粒度管理模型所制约。

2.6 可变阶数动态大页机制

上述走读暴露了 4KB 页描述符粒度的瓶颈。如果页缓存的管理粒度扩大，情况会如何变化？动态大页正是为回答这一问题而提出的。

动态大页是对传统页描述符的类型安全升级。在该机制出现之前，内核通过复合页（compound page）机制将多个连续 4KB 页组织为一个整体，但接口层面始终以页描述符指针传参——调用者拿到的指针既可能指向复合页的头页，也可能指向某个尾页，这种语义模糊迫使大量代码反复调用获取复合页头页函数来确定指针归属，增加了复杂性和出错风险。动态大页在类型层面提供了明确保证：一个动态大页指针必然指向一个完整缓存对象的头部——要么是阶数为 0 的单页，要么是复合页的头页，绝不指向尾页。这一保证消除了获取复合页头页函数的语义模糊，使编译器可以进行类型检查。

本文所讨论的动态大页指阶数大于等于 1 的页缓存对象。阶数是伙伴分配器中的概念：阶数为 0 时对应 1 个 4KB 页，阶数为 2 时对应 4 个 4KB 页即 16KB，阶数为 4 时对应 16 个 4KB 页即 64KB。当文件系统通过启用大粒度动态大页函数

(mapping_set_large_folios) 声明自身支持更大粒度的缓存对象后，页缓存在检测到对文件的持续顺序访问时，会根据预读窗口的大小和内存空闲状况动态选择阶数，分配阶数 ≥ 1 的动态大页——阶数不是预设的，而是在运行时决定的。

回到上一节的走读：在动态大页路径下，64KB 的读请求不再触发十六次分配和十六次加锁，而是由页缓存分配一个阶数为 4 的 64KB 动态大页——一次分配、一次加锁。由于动态大页对应的 64KB 物理内存是连续分配的，块设备 I/O 请求构建时可以将整个动态大页区间合并到一个 multi-page bvec 中——bv_len = 65536，bv_page 指向动态大页的首个 struct page。如果 F2FS 能够返回覆盖整个动态大页范围的连续映射，那么块层的 I/O 组装也只需围绕这个动态大页进行一次映射查询和一次区间提交——这正是动态大页回应“4KB 页缓存粒度过小”问题的核心路径。

2.7 F2FS 的异地更新、垃圾回收与压缩文件

2.4 节已说明闪存不能原地覆盖写入的根本原因：写入粒度与擦除粒度不对称，原地覆盖会触发 FTL 内部的读改写放大。F2FS 的设计直接回应了这一物理约束。

F2FS 采用日志结构文件系统的核心策略：所有数据写入都以追加方式写入当前活动日志段（active log segment）的新位置，不覆盖旧数据。

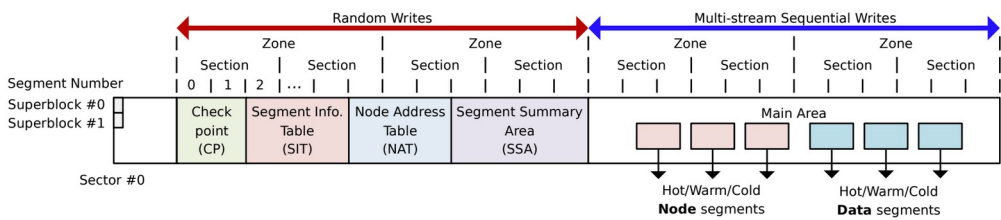


图 2-3 F2FS 磁盘布局与冷热数据分区

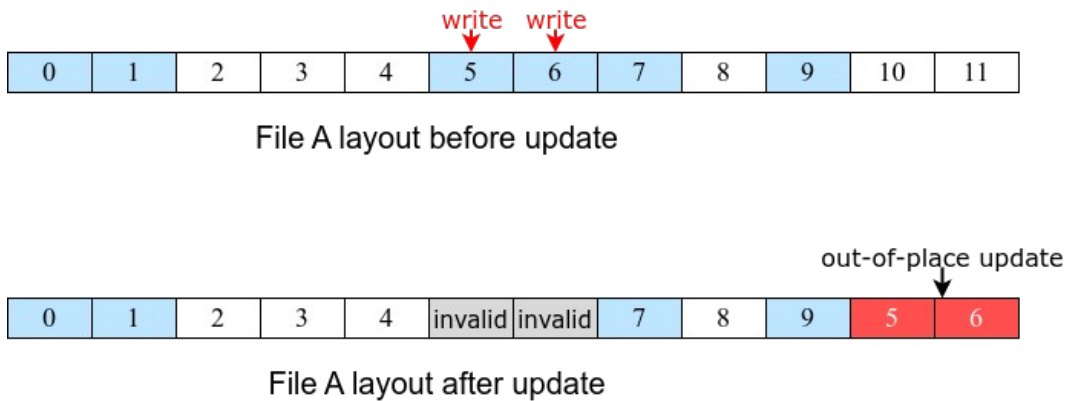


图 2-4 F2FS 异地更新过程示意

当已有的文件数据被修改时，新数据写入新的物理块，旧块随后被标记为无效。这种策略确保文件系统层发出的写入始终是顺序追加的，减少了 FTL 层面的碎片化和写放大。节点地址表（NAT）负责维护文件内部逻辑块号到当前有效物理块号的映射

关系——这正是 F2FS 实现异地更新的元数据基础。段信息表（SIT）记录每个段的块有效性和利用率，段摘要区（SSA）保存每个块的归属信息（属于哪个 索引节点 的哪个逻辑块），检查点（Checkpoint）则定期记录文件系统的一致状态以便崩溃恢复。

异地更新自然带来一个后果：随着时间推移，旧块不断变为无效，存储段中有效块和无效块混杂，需要不定期的空间回收。F2FS 的垃圾回收（Garbage Collection, GC）机制负责这一任务。GC 的基本流程是：通过 SIT 选择一个有效块比例最低的 victim 段，读取该段中仍然有效的块的数据，将其重新写入当前的活跃日志段，然后更新相应的 NAT 条目指向新位置，最后擦除 victim 段使其可重新用于分配。GC 不是一条独立的路径，它在搬移有效块时需要重新进入页缓存和文件系统映射流程——读有效块需要查询映射并构建块设备 I/O 请求，写新位置需要更新元数据。

F2FS 还支持文件数据压缩。压缩以簇（cluster）为单位，簇大小由簇大小参数（log_cluster_size）决定，通常为 2 的幂次个文件系统块。写入时，一个簇内的原始数据被压缩后以更小的体积写入磁盘；读取时，需要先将压缩数据段读入临时缓冲区，解压后再填充到目标页缓存对象中。压缩路径在操作前先通过压缩上下文（compress_ctx）将同一簇内的页缓存对象收集到一起，再统一执行压缩或解压。

上述三种机制——异地更新、GC 和压缩文件——共同决定了 F2FS 不是一条简单的“读块→写块”直线，而是普通读写、后台搬移和压缩数据处理三个路径相互交叉的网络。这为后文讨论动态大页适配的复杂性提供了必要的背景。

2.8 iomap 框架的区间映射、路径迭代与动态大页内部状态

在传统 Linux 文件系统实现中，缓冲描述符 被用来描述页内文件系统块与底层设备块的对应关系——每个文件系统块挂接一个 缓冲描述符 对象，记录其物理地址和状态。当一个页描述符对应一个块时，这种逐块组织方式足够简单；但当页缓存对象变成 64KB 的大粒度动态大页后，一个动态大页内部可能挂接十六个 缓冲描述符，遍历和维护的开销随对象变大反向恶化。图 2-5 将这种逐块 buffer_head 组织方式与 iomap 的区间化映射方式进行对照：前者围绕页内块维护分散状态，后者将文件偏移到设备地址的关系收敛为连续字节区间。

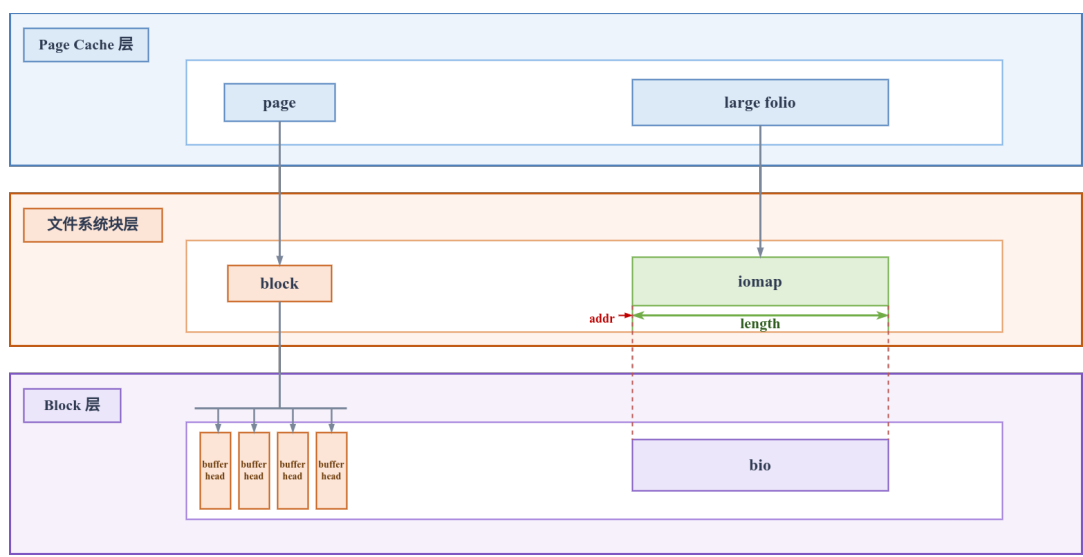


图 2-5 buffer_head 与 iomap 映射模型对比

iomap 框架通过这种根本性的思路转变来解决逐块模型的扩展性问题：将 I/O 映射的语义从“逐块”提升为“逐区间”。

iomap 的核心数据结构是 struct iomap，它描述文件的一段连续区间从文件字节偏移到设备字节地址的映射关系。一个 iomap 包含：文件内的起始字节偏移 offset、区间长度 length、对应设备的字节地址 addr、块设备指针 bdev 以及区间类型 type——如 IOMAP_MAPPED（已有映射）、IOMAP_HOLE（空洞）、IOMAP_UNWRITTEN（预分配但未写入）等。区间的长度以字节为单位描述，不需要是块大小的整数倍。表 2-3 列出了这三种区间类型及其含义。

表 2-3 iomap 区间类型

区间类型	含义
IOMAP_MAPPED	已有有效磁盘映射
IOMAP_HOLE	文件区间无数据（空洞）
IOMAP_UNWRITTEN	空间已预留但未写入（预分配）

iomap 通过迭代器模式驱动 I/O 路径。文件系统实现一组回调函数（iomap 操作集，iomap_ops）：iomap 开始回调（iomap_begin）在给定文件偏移和长度下填充 iomap 描述符，返回一段连续映射；iomap 结束回调（iomap_end）在 I/O 完成后执行收尾处理。核心迭代入口 iomap 区间迭代器（iomap_iter）循环调用 iomap 开始回调，每次获得一个区间映射后交由上层消费（读取、写入或写回），消费完当前区间后自动推进偏移，继续请求下一段映射，直到整个 I/O 请求范围被覆盖完毕。

iomap 框架向外提供了三个主要入口函数，分别对应文件系统的三条核心 I/O 路径。iomap 预读函数（iomap_readahead）服务于缓冲读路径——当页缓存检测到顺序访问模式并触发预读时，该函数通过 iomap 区间迭代器 获取映射区间，分配动态大

页并提交磁盘读取，块设备 I/O 请求完成后由 iomap 内部的 逐动态大页状态对象逐块标记 uptodate 位图。iomap 缓冲写函数（iomap_file_buffered_write）服务于缓冲写路径——应用数据先拷贝到页缓存中的动态大页页面，该函数通过 iomap 区间迭代器为写入范围获取映射区间，处理空间分配和脏区标记，写入完成后由 逐动态大页的 dirty 位图记录哪些块已被修改。iomap 写回函数（iomap_writepages）服务于脏页写回路径——当内核决定将脏页写入磁盘时，该函数遍历脏动态大页，通过 iomap 区间迭代器 为每个脏块范围查询映射，组织写块设备 I/O 请求并提交。三个入口共享同一套 iomap 操作集——文件系统只需实现一次 begin/end 回调逻辑，iomap 框架自动将其适配到读、写、写回三条路径的不同语义中。

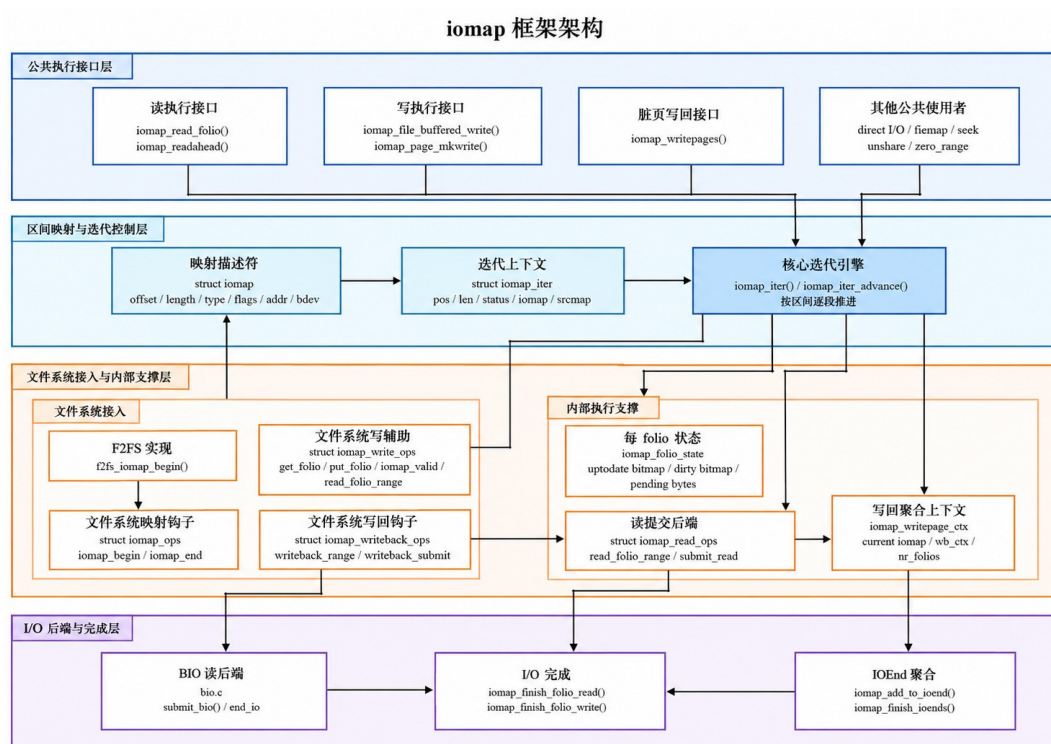


图 2-6 iomap 框架路径组织结构

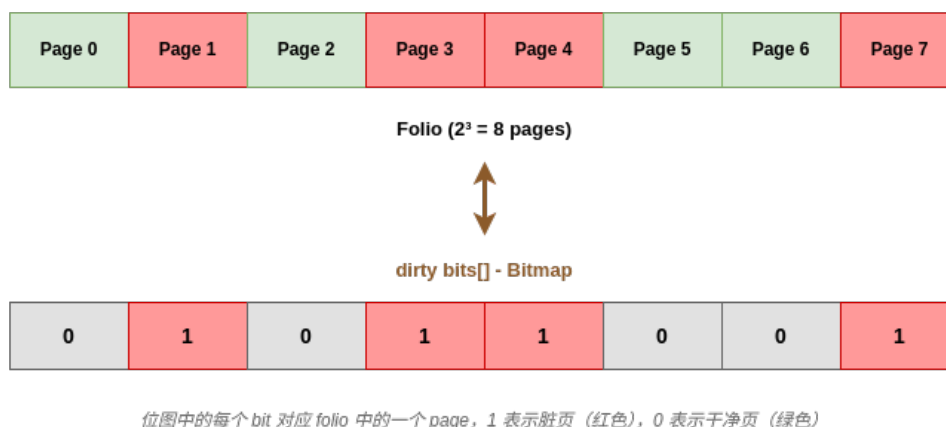
iomap 框架为每个动态大页维护一个逐动态大页状态对象，即 iomap 动态大页状态对象（iomap_folio_state），存储在动态大页的 private 字段中。它的引入直接回应了表 2-1 中"动态大页级标志只能描述整体"的局限。

当块设备 I/O 请求完成回调到达时，需要更新页缓存对象的状态。对 4KB page，回调直接设置 PG_uptodate 或清除 PG_writeback 即可；对动态大页，同一个动态大页可能被拆成多个块设备 I/O 请求、多次完成回调独立触发——对象级标志无法表达"动态大页内部哪些 4KB 子块已就绪、哪些仍在等待"。

iomap 动态大页状态对象内部包含两类信息。第一类是逐块状态位图，以文件系统块为粒度记录动态大页内每个子块是否 uptodate、是否 dirty。当一个 64KB 的动态

大页包含 16 个 4KB 块时，位图允许部分块被标记为 uptodate、部分尚未填充。第二类是 pending 字节计数：未完成读字节计数（read_bytes_pending）记录当前动态大页上尚未完成的读 I/O 字节总数，每次提交读块设备 I/O 请求时递增，完成回调中递减，仅当计数归零时读路径才解锁动态大页；未完成写回字节计数（write_bytes_pending）同理，用于写回路径，仅当计数归零时写回路径才结束 writeback。表 2-4 汇总了 iomap 动态大页状态对象的各组件。

Dirty Bits Tracking 示意图



Read_bytes_pending 示意图

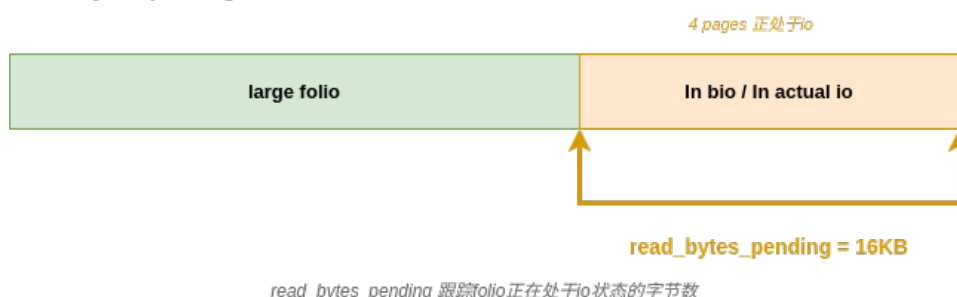


图 2-7 iomap 动态大页状态对象中的逐块位图与 pending 计数

表 2-4 iomap 动态大页状态对象 组件

组件	解决的问题	工作方式
uptodate 位图	PG_uptodate 无法区分动态大页内哪些子块已就绪	以块为粒度置位/测试
dirty 位图	PG_dirty 无法定位脏子块	写入路径仅标记实际修改的块
未完成读字节计数	单次块设备 I/O 请求完成不能判断整个动态大页读完成	提交前加 len，回调减 len，归零时动态大页读结束函数
未完成写回字节计数	单次块设备 I/O 请求完成不能判断整个动态大页写回完成	提交前加 len，回调减 len，归零时动态大页写回结束函数

2.9 本章小结

本章建立了后续讨论所需的背景知识。上半部分（2.1 至 2.5）围绕一次 64KB 缓冲读的完整路径，依次讲解了页缓存与页描述符、F2FS 块映射、块设备 I/O 请求多页 I/O 机制和闪存逻辑地址空间四个概念层，并用一个走读例子将它们串联。下半部分（2.6 至 2.8）依次讲解了三个关键技术：动态大页将页缓存对象从固定 4KB 扩展为可变阶数动态大页；iomap 以字节区间为单位组织文件 I/O 映射，并通过 iomap 动态大页状态对象的逐块位图和 pending 计数管理动态大页的内部子区间状态；F2FS 的异地更新、GC 和压缩机制则在上述通用框架之外引入了日志结构文件系统特有的路径复杂性。

第三章 F2FS 支持动态大页的关键问题分析

3.1 本章定位

前两章已经建立了本文的三层递进问题链和相应的背景知识。第一章指出，移动终端大文件访问需要扩大页缓存对象粒度，而 F2FS 的日志结构、GC 和压缩机制使得单纯的粒度扩大会触发更深层的兼容问题。第二章从页缓存、F2FS 块映射、块设备 I/O 请求和闪存 FTL 四个概念层出发，逐一说明了各层的工作机制，并在走读例子中直观展示了"上层粒度过小抵消下层大粒度 I/O 能力"的矛盾。

本章不再重复任何背景知识。它的唯一职责是：将 F2FS 已有的实现路径放到动态大页条件下逐一检验，系统指出每一层、每一条路径在动态大页进入之后具体会在哪里出问题、为什么出问题。这些问题将直接成为第四章设计与实现的问题来源。

3.2 F2FS 原 I/O 路径中的单页单块假设

在开始逐条分析之前，需要先指出一个贯穿 F2FS 全部 I/O 路径的隐含假设。这个假设在传统 4KB 路径下从未成为问题，但在动态大页条件下则成为所有问题的共同起点。它的根源是 F2FS 内部的一条宏定义：文件系统块大小被设定为恒等于系统页大小（4KB）。受此影响，F2FS 的代码中从未区分"页缓存对象的索引"和"文件系统逻辑块号"这两个概念——它们被直接当作同一个值使用。

这一假设在四条路径中的表现如下。

在普通读路径中，当页缓存请求文件系统为一个动态大页提供磁盘数据时，读取函数直接将动态大页的页缓存索引用作起始逻辑块号，以此调用块映射接口获取物理块地址，然后将整个动态大页作为一段块大小的数据加入底层的 I/O 请求中。整个过程对"一个动态大页内部可能包含多个逻辑块"没有任何准备——如果动态大页是 64KB，代码仍然只查询第一个逻辑块的映射，也只向 I/O 请求中加入头 4KB 的数据范围。

在普通写路径的写入准备阶段，代码同样将动态大页的页缓存索引直接作为逻辑块号，通过一次节点树查找获取该块的当前物理地址或进行空间预分配，然后以这个单一块地址为目标发起磁盘读取以填充待写入的动态大页。整个过程对“同一个动态大页覆盖了多个逻辑块、每个块需要独立的地址查询和状态判断”没有预留任何处理分支。

在脏页写回路径中，写回入口函数首先将每个动态大页拆解为独立的页描述符数组——对一个 64KB 的动态大页就是十六个独立的页被放入数组——然后逐页调用同一个单页写回函数。该函数内部以动态大页的起始索引作为块坐标来组织映射查询和 I/O 提交。在传统路径中，一个动态大页就是一个页、一个页就对应一个块，所以拆解后每个页被正确地写回它对应的唯一的一个块。但当动态大页包含十六个块时，拆解出的十六个页全部共享同一个起始索引——每一次单页写回调用都会将同一个起始块重复提交，而不是按照子页在动态大页内的偏移依次推进到不同的逻辑块。

在 GC 搬移路径中，GC 的核心搬移函数以单个逻辑块号为输入参数，在页缓存中查找该块对应的动态大页，然后将其交给写回函数执行实际的搬移写入。写回函数内部以动态大页的起始页缓存索引作为块坐标来查询节点树并确定写入的物理位置。在传统路径中，输入的逻辑块号恰好就是动态大页的起始索引——一个动态大页只覆盖一个块，这两个值永远是同一个数。但当 GC 请求搬移的逻辑块落在一个动态大页的覆盖范围之内时，该动态大页的起始索引与目标块的逻辑块号不再相等——例如一个 64KB 的动态大页起始索引为 0，而 GC 要搬移的逻辑块号可能是 5。写回函数以索引 0 而不是 5 来定位节点树条目，导致将错误位置的块数据写到搬移目标，而不是实际需要搬移的那个块。

上述四条路径的共性在于：它们从不同角度出发——读取、写入准备、写回提交、后台搬移——最后都默认了一个动态大页只管理一个文件系统块。动态大页使这个假设不再成立。

3.3 F2FS 私有标志与 iomap 状态对象的字段冲突

3.2 节展示了 F2FS 全路径中单页单块假设的具体失效形式，本节转向另一个同样由“一个对象=一个块”前提破裂引发的困难：页缓存对象私有字段的语义冲突。

每个页缓存对象预留了一个名为 `private` 的字段（泛型指针类型），供文件系统自由使用。在传统 4KB 路径中，F2FS 直接利用这个字段的低位保存各类私有标志——例如当页正被 GC 搬移时，设置 GC 标记函数（`set_page_private_gcging`）将正在迁移标志（`ONGOING_MIGRATION`）写入该字段；当页属于原子写上下文时，写入原子写标志。这些标志使用小整数编码，和指针在二进制层面的表示完全不同。

问题在于，`iomap` 框架在缓冲 I/O 路径中也依赖同一个 `private` 字段。`iomap` 为每个动态大页挂接一个 逐动态大页状态对象（`iomap` 动态大页状态对象），通过动态大

页的 `private` 字段保存指向该对象的指针。状态对象内部包含该动态大页每个文件系统块的就绪和脏位图，以及未完成读写字节的原子计数。读完成路径通过解引用动态大页的 `private` 字段获取指针后更新位图和计数；写回路径同样依赖这些位图来定位动态大页内的脏区。

两条路径对同一个字段赋予互不相容的解释。崩溃路径是直接的：若 F2FS 先通过 GC 路径将 `0x1`（表示正在迁移标志）写入动态大页的 `private` 字段，随后该对象进入 `iomap` 的读完成路径——`iomap` 完成动态大页读函数（`iomap_finish_folio_read`）将动态大页的 `private` 字段的值当作 `iomap` 动态大页状态对象指针解引用，CPU 试图访问地址 `0x1` 附近的内存区域，直接触发内核 `panic`。反之亦然——若 `iomap` 先挂接了状态对象指针，F2FS 的 GC 路径再将其当作小整数标志做位测试，同样会产生无意义的结果。

这一冲突并非只影响高阶动态大页。0 阶动态大页（大小仍为 4KB）同样可能交替进入 F2FS 的 GC 或原子写路径和 `iomap` 的读写路径——只要它经过了 `iomap` 缓冲 I/O，其 `private` 字段就已经被 `iomap` 接管。这意味着所有可能进入 `iomap` 路径的动态大页——无论阶数是多少——都必须将 F2FS 私有标志、`iomap` 位图状态和运行时计数纳入同一对象。

3.4 动态大页跨越压缩簇与非压缩区间的问题

F2FS 的压缩文件以簇为基本单位——每若干连续文件系统块（通常为 4 块）组成一个簇，压缩和写出均以簇为单位进行。在传统 4KB 路径中，每个页缓存对象就是一个独立的 4KB 页，它要么全部属于某个簇，要么全不属于。压缩路径对这个前提有着深层的依赖：脏页以页为单位被收集进压缩上下文，同一个簇内的页被逐页标记写回、逐页提交、逐页解锁——整个流程默认“一个页就是一个完整的独立生命周期对象”。

动态大页首先在锁和生命周期层面打破了这个前提。一个 64KB 的动态大页包含十六个子页，这些子页可能分散在多个压缩簇中——例如前四个页属于簇 A，中间八个页属于簇 B 和 C，后四个页属于簇 D。压缩路径处理簇 A 时，对属于该簇的四个子页逐一执行加锁、标记写回、提交和解锁操作——但解锁的对象是同一个动态大页。

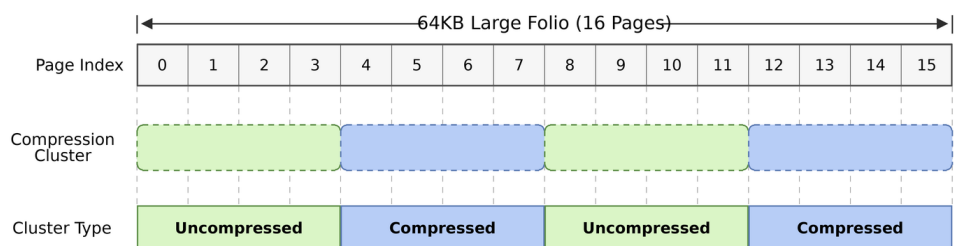


图 3-1 动态大页跨越多个压缩簇的生命周期冲突

簇 A 的压缩写出尚未完成，动态大页已经被解开：任何其他线程都可能在此刻进入这个半就绪状态的对象。当压缩路径随后处理簇 B 时，会对同一个动态大页再次尝试加锁——此时锁虽然成功获取，但动态大页内部状态已被簇 A 的释放动作扰乱：簇 B 以为自己在操作一个完整的、处于 `writeback` 状态的对象，实际上这个对象的锁已经在不同簇之间被反复释放和重新获取。对于动态大页而言，压缩路径中默认的“逐页加锁→逐页解锁”循环不再等价于“每个对象的锁周期是完整且互斥的”。

同样的冲突在写回完成时机上表现得更隐蔽。压缩数据段全部写回磁盘后，完成回调遍历压缩上下文中该簇的所有页，对每个页调用写回结束标记。但对于动态大页，同一个动态大页的子页被不同簇的完成回调各自处理——簇 A 的所有压缩页对应的块设备 I/O 请求先完成，回调遍历簇 A 的页数组，为动态大页调用一次写回结束标记。但此时动态大页的其他子页仍处于簇 B、簇 C、簇 D 的压缩块设备 I/O 请求队列中。一旦这个提前的完成标记被上层感知，该动态大页可能被回收、分配或在其他路径中被当作已完成写回的干净对象使用——而实际仍有脏数据未落盘。如第二章所述，块设备 I/O 请求的完成回调在中断上下文中异步执行，各回调之间互不感知——这正是第二章 2.3 节分析的异步回调机制在压缩路径中的直接映射：一个动态大页的多个子范围被拆分到多个块设备 I/O 请求和多个簇中，各回调独立触发，但没有任何机制来汇总“整个动态大页的写回是否全部完成”。

在读路径上，压缩文件的困难形态不同但根源相同。一个动态大页的不同子范围可能同时需要解压填充（覆盖在压缩簇内的部分）和直接磁盘读（覆盖在非压缩块上的部分）。两种 I/O 来源的完成路径不同——一个是解压完成后的回调填充就绪位图，一个是块设备 I/O 请求完成后的回调标记就绪——但两者的终点是同一个动态大页。读完成的判定必须跨越这两种异步来源，在所有子范围都就绪之后才能解锁并标记整个动态大页读取完成。

3.5 脏页写回中的脏区丢失与块索引错位

在传统 4KB 路径中，脏页写回的语义是清晰的：一个页的 `PG_dirty` 标志表示该页中有数据被修改，写回路径围绕该页查询块映射、组织块设备 I/O 请求并提交到底层设备。页和文件系统块大小一致——dirty 标记的边界与 I/O 提交的边界在 4KB 配置下恰好重合，一个页脏了，提交的就是一个 4KB 块。

动态大页使这一重合在两个层面上同时断裂。

第一个层面是脏区信息的丢失。F2FS 原有的写回入口函数从页缓存中取出动态大页后，通过调用清除动态大页脏标记函数（`folio_clear_dirty_for_io`）清除整个动态大页的 `PG_dirty` 标志——无论这个动态大页内部只有一个子块被修改，还是全部十六个子块都被修改。一旦全动态大页的 `dirty` 被清除，写回路径就失去了区分“哪些子块需要实际写回、哪些子块无需写回”的信息。对于日志结构的 F2FS 而言，这意味着一次局部修改可能迫使整个动态大页进入写回流程，产生远超实际需要的日志追加写入——应用只需要写回 4KB，日志却接收了 64KB 的新数据。这 60KB 的额外写入不仅消耗日志空间，加速当前段的填满，还会在后续 GC 中转化为额外的有效块搬移，形成连锁的写放大。而对于通过擦除数据块来进行新块地址更新的闪存设备而言，写放大就意味着损耗变大、寿命下降。

第二个层面更为直接：写回路径在将动态大页拆解为页数组后，对数组中的每个页条目逐一调用同一个单页写回函数。该函数内部以动态大页的起始页缓存索引作为块坐标，来查询节点树并确定写入的物理位置。对于传统单页对象，页条目和动态大页起始索引恰好相等——拆解出的页条目就是动态大页本身，索引自然指向正确的逻辑块。但对于 64KB 动态大页，十六个页条目共享同一个动态大页起始索引——例如起始索引为 0 的动态大页，拆解出十六个页条目，但写回函数被调用十六次，每次都以索引 0 去查询节点树。结果是同一逻辑块被反复写入了十六次，而实际覆盖逻辑块 1 到 15 的子页数据从未被写入到对应的逻辑位置。

3.6 本章小结

本章以 F2FS 全路径中“一个页缓存对象等于一个文件系统块”的默认假设为起点，在四条路径中逐一展示了这一假设的具体形态（3.2 节）。基于这一共同前提，本章从两个递进层面分析了动态大页进入后的问题。第一个层面是 F2FS 私有状态与 `iomap` 逐动态大页状态在同一个 `private` 字段上的所有权冲突（3.3 节，对应 Layer 3）。第二个层面是该冲突在 F2FS 特有机制中的传导与放大：压缩文件的簇边界与动态大页边界不重合（3.4 节），以及脏页写回的粒度失控与写放大（3.5 节）。

第四章 F2FS 动态大页缓冲 I/O 设计与实现

4.1 总体架构

F2FS 动态大页缓冲 I/O 的实现复用通用 iomap 框架，接入 F2FS 定制实现的钩子函数，并对 F2FS 专属路径（GC 以及压缩）进行框架扩展。图 4-1 给出了各组件之间的关系。

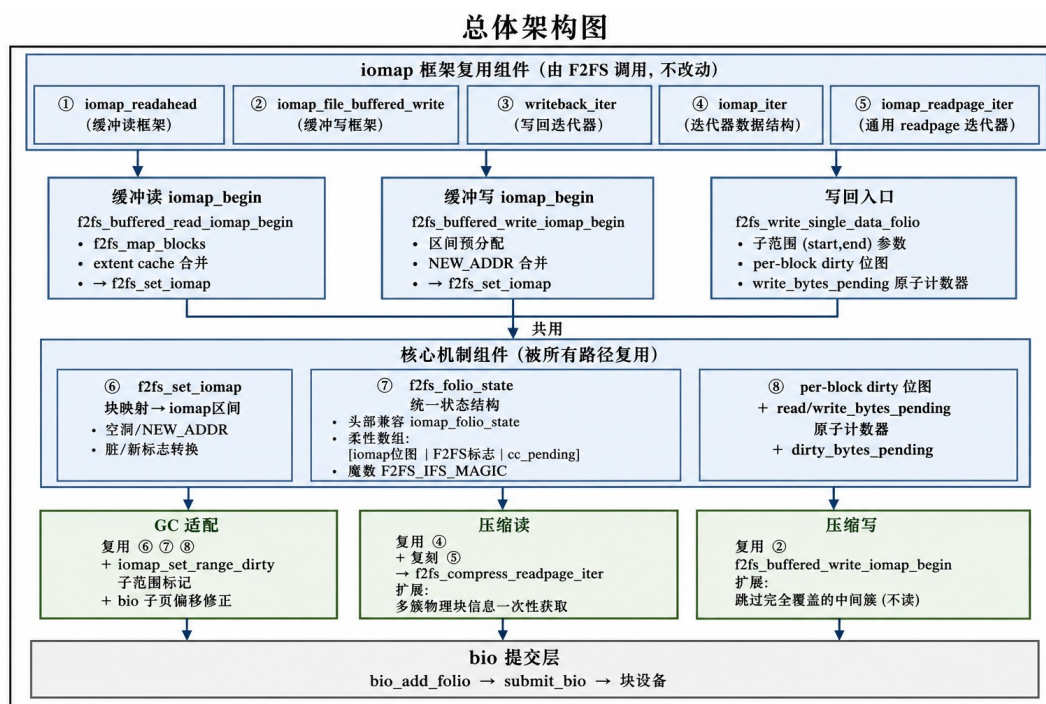


图 4-1 F2FS 动态大页缓冲 I/O 总体架构

F2FS 在缓冲读路径中复用 iomap 预读框架（①），通过自己的区间映射回调（⑥）将块级结果转换为字节区间；在缓冲写路径中复用 iomap 缓冲写框架（②），通过区间预分配回调提供连续空间预留；在写回路径中复用写回迭代器（③），由自定义的单个动态大页写回函数执行子范围写回提交至块设备层。上述路径共同依赖 F2FS 区间设置函数（⑥）完成块映射到 iomap 区间的转换，并依赖 F2FS 扩展型动态大页状态对象（⑦）保存逐块位图、F2FS 私有标志和完成计数（⑧）。

GC 搬移保留单块语义，复用 ⑥⑦⑧，通过子范围 dirty 标记和块设备 I/O 请求偏移修正定位动态大页内部的目标块。压缩读复用 iomap 区间迭代器（iomap_iter，④）的外层推进逻辑，以 F2FS 压缩读页迭代器（f2fs_compress_readpage_iter）替换通用读迭代器（⑤）内部的提交策略，并扩展多簇物理块信息一次性获取。压缩写复用缓冲写的区间准备流程，在完整覆盖的中间簇上跳过旧数据读取。所有路径最终以块设备 I/O 请求提交到底层块设备。

为便于后续各路径之间进行统一对比，本章主要以 64KB 动态大页作为说明示例。

4.2 普通文件的区间映射 I/O

4.2.1 块级映射到字节区间的转换

F2FS 的块映射接口以逻辑块号 `m_lblk`、物理块号 `m_pblk` 和连续块数 `m_len` 为单位返回映射结果，而 `iomap` 框架以文件字节偏移 `offset`、设备字节地址 `addr` 和字节长度 `length` 描述映射区间。当动态大页在 I/O 路径中的覆盖面从 1 块扩展到 16 块后，原先隐藏在逐块调用中的语义分歧逐一显现：各路径对 F2FS 块映射函数返回结果的解释方式不统一，空洞边界、新分配区间和已映射块的类型判定逻辑分散在多处调用点中，任何一处不一致都会导致区间类型误判。块映射到字节区间的转换涉及长度单位的换算、空洞终点判定、预分配区间的未写语义保留和脏状态传递——这些规则若继续分散在读、写和写回路径中，动态大页覆盖的多块区间就会被拆成不一致的 I/O 语义。

为此，本系统将块映射到字节区间的转换逻辑收敛到单一函数 F2FS 区间设置函数中。该函数同时服务于缓冲读、缓冲写和直接 I/O 三条路径，其核心逻辑如算法 4-1 所示。

算法 4-1 f2fs_set_iomap: block map to iomap

```
1  Input: inode, map, offset, length, dio
2  Output: iomap
3  iomap.offset <- F2FS_BLK_TO_BYTES(map.m_lblk)
4  if map has F2FS_MAP_MAPPED:
5      iomap.length <- F2FS_BLK_TO_BYTES(map.m_len)
6      if dio and map.m_pblk == NEW_ADDR:
7          return -EINVAL
8      if map.m_pblk != NEW_ADDR:
9          iomap.type <- IOMAP_MAPPED; set IOMAP_F_MERGED
10         iomap.addr <- F2FS_BLK_TO_BYTES(map.m_pblk)
11     else:
12         iomap.type <- IOMAP_UNWRITTEN; iomap.addr <- IOMAP_NULL_ADDR
13 else:
14     if dio and write flag set:
15         return -ENOTBLK
16     if map.m_pblk == NULL_ADDR:
17         iomap.length <- hole_end(map) - iomap.offset
18         iomap.type <- IOMAP_HOLE; iomap.addr <- IOMAP_NULL_ADDR
19     elseif map.m_pblk == NEW_ADDR:
20         iomap.length <- F2FS_BLK_TO_BYTES(map.m_len)
```

```

21         iomap.type <- IOMAP_UNWRITTEN; iomap.addr <- IOMAP_NULL_ADDR
22     else:
23         return -EFAULT
24     if map has F2FS_MAP_NEW:
25         set IOMAP_F_NEW
26     if inode is datasync-dirty or offset + length > i_size(inode):
27         set IOMAP_F_DIRTY
28     return 0

```

该函数在三种映射状态间做出精确区分。当 F2FS_MAP_MAPPED 已置位时，进一步检查 m_pblk：若为有效物理块号则输出 IOMAP_MAPPED 类型并设置 IOMAP_F_MERGED 标志（表示该 iomap 覆盖的是一个连续映射区间）；若为 NEW_ADDR 则输出 IOMAP_UNWRITTEN（延迟分配尚未落盘）。当映射未命中时，m_pblk == NULL_ADDR 表示空洞，此时通过 F2FS_MAP_NODNODE 区分两种空洞：若该位置无直接节点条目，空洞终点以 m_next_pgofs 为界（即下一个存在节点块的逻辑页号）；若有节点条目但映射为空，则以 m_len 为界。最后，F2FS_MAP_NEW 和索引节点 脏状态分别被转换为 IOMAP_F_NEW 和 IOMAP_F_DIRTY 标志。

引入该转换层之前，F2FS 原生读接口每次只请求单个块的映射（map.m_len = 1）。对一个 64KB（16 块）的动态大页，这意味着 16 次 F2FS 块映射函数 调用和 16 次独立的块映射结果解释。转换层建立后，上层的 iomap 开始回调将整个区间请求一次性传递给 F2FS 块映射函数，后者内部的 块合并判断函数 逻辑自动合并物理和逻辑连续的相邻块，使 map.m_len 扩大到覆盖整个连续区间。F2FS 区间设置函数 随后将这一扩展后的结果一次性转换为 iomap 字节区间。在连续映射能够一次返回时，映射调用从 16 次降至 1 次，锁操作次数同步下降。

4.2.2 普通文件缓冲读路径

F2FS 单页读函数 每次调用仅向 F2FS 块映射函数 请求一个逻辑块的映射，然后将其单独加入块设备 I/O 请求。当动态大页将单次覆盖范围从 1 块扩展到 16 块后，这一路径的映射调用次数、锁操作次数和块设备 I/O 请求提交次数同步放大 16 倍。

F2FS 缓冲读回调（算法 4-2）将动态大页覆盖的字节范围一次性转换为 F2FS 的逻辑块区间，交由 F2FS 块映射函数 内部自动合并连续块，最后通过 F2FS 区间设置函数 生成 iomap 区间描述符。

算法 4-2 f2fs_buffered_read_iomap_begin: range map for buffered read

```

1 Input: inode, offset, length, flags
2 Output: iomap

```

```

3 map.m_lblk <- F2FS_BYTES_TO_BLK(offset)
4 map.m_len <- F2FS_BYTES_TO_BLK(offset + length - 1) - map.m_lblk + 1
5 map.m_next_pgofs <- &next_pgofs; map.m_may_create <- false
6 if flags has IOMAP_WRITE:
7     return -EINVAL
8 err <- call f2fs_map_blocks(inode, &map, F2FS_GET_BLOCK_IOMAP)
9 if err != 0:
10     return err
11 if map.m_pblk == COMPRESS_ADDR:
12     return -EINVAL
13 return call f2fs_set_iomap(inode, &map, iomap, flags, offset, length,
    false)

```

该回调的核心操作是两次转换：先将字节区间 (`offset, length`) 转换为逻辑块号区间 (`map.m_lblk, map.m_len`)，在 F2FS 块映射函数 返回后通过 4.2.1 节的 F2FS 区间设置函数 将块映射结果转换为 `iomap` 区间描述符。其中的关键标志是 `F2FS_GET_BLOCK_IOMAP`——它告知 F2FS 块映射函数 内部遍历循环在块映射信息足以构建 `iomap` 时立即返回，而非遍历到直接节点页末尾，从而将返回结果的粒度控制在“连续映射子区间”水平。

F2FS 块映射函数 在收到区间请求后，在内部遍历逻辑块并调用 块合并判断函数 合并物理和逻辑均连续的相邻块——(`map->m_pblk + map->m_len == blkaddr`) && (`map->m_next_extent == pgofs`)——每合并一个块，`m_len` 递增 1。遍历结束后将合并后的连续区间信息缓存，供后续请求直接命中。

在 F2FS 区间设置函数 填充好 `iomap` 之后，剩余工作由 `iomap` 框架的 `iomap` 区间迭代器自动完成：它根据 `iomap` 长度推进迭代位置，每次迭代调用 `iomap` 读页迭代器将当前区间内的页面加入块设备 I/O 请求、管理 `uptodate` 位图，并在整个动态大页的所有子区间全部就绪后解锁。文件系统不需要关心块设备 I/O 请求构建、完成回调注册或解锁时机的具体实现。

表 4-1 原生路径与本系统路径读路径对比

指标	原生路径（F2FS 单页读）	本系统路径（F2FS 缓冲读回调）
F2FS 块映射 调用次数	16 次（逐块）	1 次（连续区间）或 N 次（N=连续子区间数）
映射请求粒度	每次 1 块	一次整个字节区间
块合并	不适用（每次只请求 1 块）	块合并判断 遍历中自动合并
dnode 锁操作	16 次	1 次

块设备 I/O 请求提交	16 次独立提交	iomap 框架自动合并为更少提交
--------------	----------	-------------------

4.2.3 普通文件缓冲写路径

F2FS 原生写入准备函数围绕单个逻辑块工作，一次只分配一个空闲块（F2FS 预留新块函数），所有元数据操作——**dnode** 查找、段锁获取、空间预留——均为逐块执行。当动态大页使单次写入的块数从 1 上升到 16 后，这种"分配一个、写入一个、再分配下一个"的循环使元数据操作开销与块数线性增长，且底层块合并能力（块合并判断函数）因上层每次只请求 1 块而完全闲置。

针对这一瓶颈，本系统用 F2FS 缓冲写回调（算法 4-3）将逐块分配替换为区间级预分配。

算法 4-3 f2fs_buffered_write_iomap_begin: range allocation for buffered write

1	Input: inode, offset, length, flags
2	Output: iomap
3	map.m_lblk <- F2FS_BYTES_TO_BLK(offset)
4	map.m_len <- F2FS_BYTES_TO_BLK(offset + length - 1) - map.m_lblk + 1
5	map.m_may_create <- true
6	iomap.bdev <- inode.sb.bdev; iomap.folio_ops <- f2fs_iomap_folio_ops
7	err <- call f2fs_map_blocks(inode, &map, F2FS_GET_BLOCK_IOMAP)
8	if err != 0 and err != -ENOSPC:
9	return err
10	if map.m_pblk == NULL_ADDR:
11	err <- call f2fs_map_blocks_preallocate(inode, &map)
12	if err != 0 and err != -ENOSPC:
13	return err
14	return call f2fs_set_iomap(inode, &map, iomap, flags, offset, length, false)

该回调分两个阶段。第一阶段以 F2FS_GET_BLOCK_IOMAP 模式调用 F2FS iomap 块映射函数，查询目标写入区间内是否已有物理块映射。第二阶段仅在区间为空洞时触发：调用 F2FS 预分配块映射函数，内部以 F2FS_GET_BLOCK_PRE_AIO 标志再次驱动 F2FS 块映射函数，在遍历到直接节点页末尾时调用 F2FS 批量预留新块函数 进行批量空间预留——一次调用为整个写入区间分配连续空闲块，返回一个横跨多个逻辑块的、以 NEW_ADDR 为物理地址的统一映射。

批量预分配能生效的关键在于 块合并判断函数 对 NEW_ADDR 的特殊处理：当前块和前一映射都是 NEW_ADDR 时，合并条件同样成立——(map->m_pblk ==

blkaddr && blkaddr == NEW_ADDR)。这使连续的新分配区间被合并为一个大区间返回，而非退化为每次一个块的细粒度分配。

在 iomap 框架层面，写路径享有两项额外收益。动态大页生命周期管理由框架自动完成：iomap 写入开始函数负责查找或创建动态大页并加锁，文件系统回调在安全的已锁定上下文中执行，无需手动处理并发截断竞争。脏区标记精度也得到提升：iomap 写入结束函数通过 iomap_set_range_dirty(folio, offset_in_folio, copied) 将动态大页内被实际修改的字节区间标记为脏，而非整动态大页标记——这对后续 4.5 节的子范围写回控制至关重要。

表 4-2 原生逐块写准备路径与区间预分配路径对比

指标	原生路径（F2FS 写入准备）	本系统路径（F2FS 缓冲写回调）
空间分配	逐块分配，每次 1 个 NEW_ADDR	区间预分配，一次 N 个 NEW_ADDR
dnode 查找	N 次	1 次
F2FS 预留新块	N 次	1 次（内部批量）
连续块合并	每次 1 块，无法合并	块合并判断 合并连续 NEW_ADDR
原子文件	独立复制 dnode 逻辑	复用 F2FS 块映射_iomap，仅切换 索引节点

4.3 F2FS 扩展型动态大页内部状态对象

在动态大页条件下，页缓存对象的状态需要拆成两个层次理解。动态大页级标志（PG_locked、PG_uptodate、PG_dirty、PG_writeback）仍然描述整个对象是否被锁定、是否就绪、是否脏或是否正在写回；但一个动态大页内部包含多个文件系统块，不同子块可能处于不同的 uptodate、dirty 或 I/O 未完成状态。例如，一个 64KB 的动态大页覆盖 16 个 4KB 块，其中前 4 个块已从磁盘读入且被修改，中间 8 个块尚未读取，后 4 个块已读入但未修改——动态大页级标志无法区分这 16 个子块各自的状态。

iomap 框架引入 iomap 动态大页状态对象，即动态大页内部子区间状态对象。该对象挂接在动态大页的 private 字段上，内部用逐块位图记录动态大页内哪些文件系统块已经 uptodate 或 dirty，并用 pending 字节计数汇总多个异步 I/O 完成事件。具体而言，未完成读字节计数记录当前动态大页上尚未完成的读 I/O 字节总数，仅当计数归零时读路径才解锁动态大页；未完成写回字节计数记录尚未完成的写回字节总数，仅当计数归零时写回路径才结束 writeback。一个动态大页可拆成多个块设备 I/O 请求、多个子范围异步处理，完成事件的汇总由计数器统一驱动。

这个状态对象解决了动态大页内部子区间状态表达问题，却与 F2FS 原有的动态大页 `private` 字段用法发生冲突。F2FS 原本也使用该字段直接编码 `ONGOING_MIGRATION`、`ATOMIC_WRITE` 等整数标志——如果同一字段被两条路径分别按指针和整数解释，跨路径流动的动态大页就会出现语义冲突：GC 路径写入的 `ONGOING_MIGRATION` 标志被 `iomap` 读完成回调当作 `iomap` 动态大页状态对象指针解引用，直接触发内核 `panic`。0 阶动态大页同样面临此风险——GC 路径向 0 阶动态大页写入标志后，该动态大页可能随后进入 `iomap` 缓冲写路径。

本系统因此设计了 F2FS 扩展型动态大页状态对象。它保留 `iomap` 动态大页状态对象的头部布局，使 `iomap` 能继续按原方式访问位图和 `pending` 计数，同时在后续空间中扩展 F2FS 私有标志和压缩路径所需的脏 `pending` 计数。

字段/代码片段
<code>struct f2fs_folio_state {</code>
<code>spinlock_t state_lock;</code>
<code>unsigned int read_bytes_pending;</code>
<code>atomic_t write_bytes_pending;</code>
<code>unsigned long state[]; // 柔性数组</code>
<code>};</code>

F2FS 扩展型动态大页状态对象与 `iomap` 动态大页状态对象的关系可以从两个维度理解。在内存布局上，前三成员——`state_lock`、未完成读字节计数、未完成写回字节计数——与 `iomap` 动态大页状态对象完全相同，因此 `iomap` 通用函数以 `(struct iomap_folio_state *)folio->private` 访问这些字段时，偏移量和语义完全等价。在功能上，F2FS 扩展型动态大页状态对象在 `iomap` 动态大页状态对象的逐块位图之后，通过柔性数组 `state[]` 额外分配两个 `unsigned long`：`state[iomap_longs]` 存放 F2FS 私有标志（`ONGOING_MIGRATION`、`ATOMIC_WRITE`、`DEFERRED_UNLOCK` 等），`state[iomap_longs+1]` 作为 未完成脏字节计数 原子计数器，供压缩文件写回路径使用（4.5.2 节）。

由于动态大页的 `private` 字段可能指向 `iomap` 动态大页状态对象或 F2FS 扩展型动态大页状态对象，运行时必须区分两者。本系统在分配 F2FS 扩展型动态大页状态对象时，将魔数 `F2FS_IFS_MAGIC` 写入 未完成读字节计数字段——该字段在 `iomap` 框架中仅在读 I/O 进行时才携带真实计数值，初始化后处于空闲状态。类型检查函数 `F2FS` 类型检查函数通过读取该字段并与魔数比对完成识别。

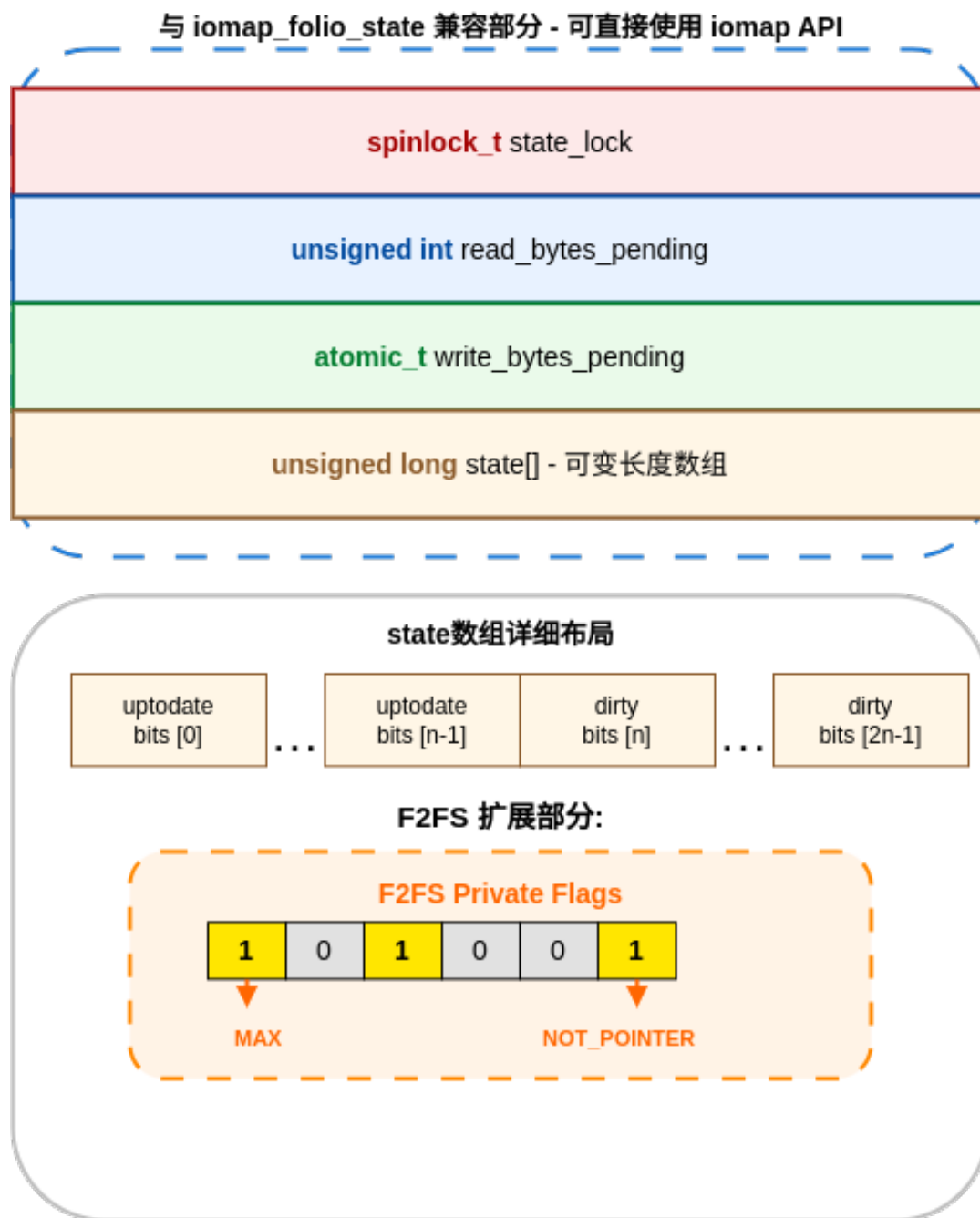


图 4-2 F2FS 扩展型动态大页状态对象布局

结构体的核心分配函数 F2FS 状态分配函数（算法 4-4）处理三种边界情况。

算法 4-4 f2fs_ifs_alloc: attach or migrate state

```

1 Input: folio, gfp, force_alloc
2 Output: f2fs_folio_state or NULL
3 old <- folio_get_private(folio)
4 if folio_order(folio) == 0 and not force_alloc:
5     return NULL
6 if old != NULL:

```



```

7      rbp <- READ_ONCE(old.read_bytes_pending)
8      if rbp >= F2FS_IFS_MAGIC:
9          return old
10
11     new <- call kmalloc(sizeof(f2fs_folio_state) +
12     f2fs_ifs_total_longs(folio))
13
14     if new == NULL:
15         return NULL
16
17     call ifs_to_f2fs_ifs(old, new, folio)
18     init counters and state_lock
19     call folio_change_private(folio, new)
20     call kfree(old)
21     return new
22
23 new <- call kzalloc(sizeof(f2fs_folio_state) +
24 f2fs_ifs_total_longs(folio))
25
26 if new == NULL:
27     return NULL
28
29 init counters and state_lock
30
31 if folio_test_uptodate(folio):
32     set all uptodate bits
33
34 if folio_test_dirty(folio):
35     set all dirty bits
36
37 call folio_attach_private(folio, new)
38
39 return new

```

第一种，动态大页的 `private` 字段为空：直接分配并初始化新的 F2FS 扩展型动态大页状态对象，根据当前动态大页的 `uptodate/dirty` 状态初始化对应位图。第二种，动态大页的 `private` 字段已指向 `iomap` 动态大页状态对象（魔数不匹配）：分配更大的 F2FS 扩展型动态大页状态对象，将旧的逐块位图内容通过 F2FS 状态转换函数拷贝到新结构体对应位置，写入魔数，通过动态大页字段替换函数原子替换指针并释放旧结构体。第三种，魔数匹配：直接返回已有指针。

为兼容 F2FS 原有的大量设置 GC 标记函数、测试原子写标记函数等调用点，本系统设计了新的宏族 `F2FS_FOLIO_PRIVATE_GET/SET/CLEAR_FUNC`。以 `Get` 操作为例：首先检查动态大页的 `private` 字段是否已设置

`PAGE_PRIVATE_NOT_POINTER` 标志——若是，说明存储的是旧式直接标志位，直接在该值上进行位操作（快速路径，兼容元数据动态大页）；若否，说明是指针，则检查魔数后通过 F2FS 私有标志指针定位到柔性数组中的私有标志位区域进行位操作。Set 操作的分配策略通过 F2FS `iomap` 判断函数判断当前动态大页是否可能进入 `iomap`

缓冲写路径——若可能（由 force_alloc 参数体现），即使该动态大页当前为 0 阶且尚未有任何私有状态，也必须通过 F2FS 状态分配函数分配状态对象。

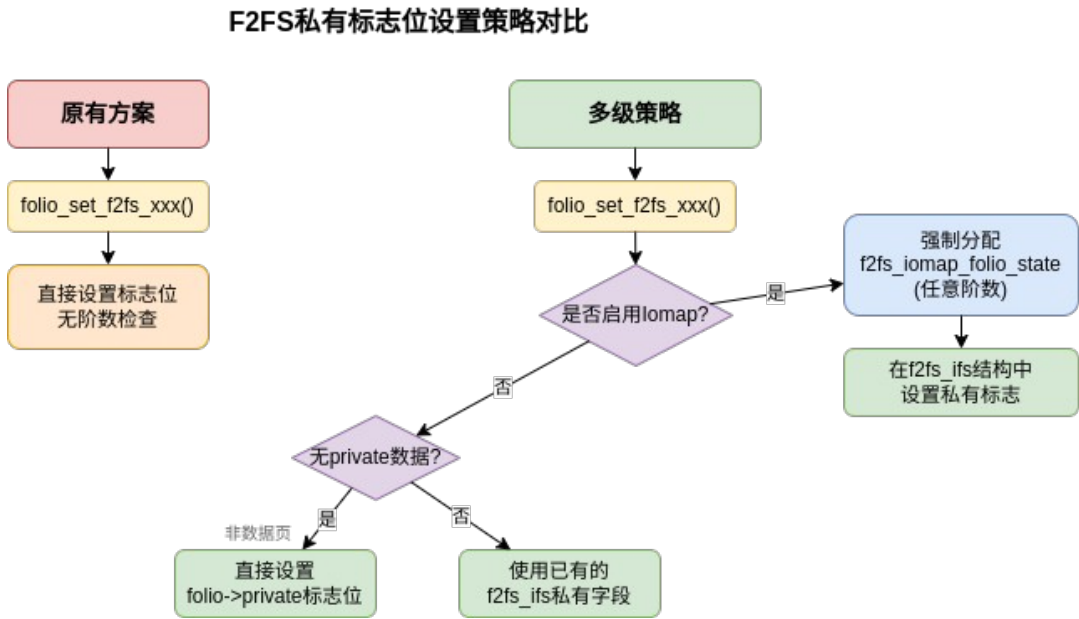


图 4-3 F2FS private 字段兼容策略

4.4 压缩文件的动态大页读写适配

压缩文件路径的特殊性在于数据以压缩簇（通常 4 块 16KB）为单位组织，而动态大页的覆盖范围可能与簇边界不重合——一个 64KB 的动态大页可能跨越 4 个 16KB 压缩簇，不同子范围可能分别需要解压填充和直接磁盘读。本节处理压缩文件的两条前台路径：buffered read 和 buffered write。压缩文件的写回路径留至 4.5 节，与普通文件写回对照处理。

4.4.1 压缩文件缓冲读适配

压缩文件读取存在混合 I/O 模型：一个动态大页的不同子范围可能位于压缩簇内（需要"读取压缩块→解压到动态大页"的间接路径）或位于非压缩区域（需要"磁盘→动态大页"的直接读路径）。两条路径的完成时机不同——解压由 CPU 异步执行，块设备 I/O 请求完成由中断上下文触发——若无法统一协调，就会出现动态大页在部分子范围仍未就绪时被提前解锁。图 4-4 用同一动态大页跨越多个压缩簇的布局说明这一问题：同一个动态大页内部的子页可能分别落入不同簇，其完成事件必须统一汇总。

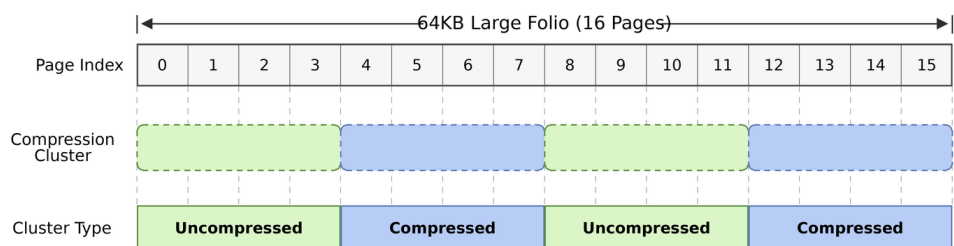


图 4-4 压缩路径中动态大页跨簇子区间示意

本系统为压缩文件缓冲读实现了一套完整的 `iomap` 适配路径，由三个组件构成：簇感知的 `iomap` 开始回调、替代 `iomap` 读页迭代器 的内层迭代器、以及统一的完成计数机制。

`iomap` 开始回调：从单簇到多簇。F2FS 压缩读回调（算法 4-5）是压缩读的 `iomap` 迭代入口。其核心改进在于增加了多簇处理路径。

算法 4-5 `f2fs_buffered_read_compress_iomap_begin: compressed-cluster mapping`

```

1  Input: inode, pos, length, flags
2  Output: iomap
3  ctx <- iter.private
4  index <- pos >> PAGE_SHIFT
5  folio <- ctx.cur_folio or call readahead_folio(ctx.rac)
6  cluster <- cluster_idx(ctx.cc, index)
7  all <- (folio_nr_pages(folio) - (index - folio.index)) >>
   ctx.cc.log_cluster_size
8  iomap.offset <- F2FS_BLK_TO_BYTES(F2FS_BYTES_TO_BLK(pos))
9  iomap.bdev <- inode.sb.bdev
10 if index != cluster << log_cluster_size or all <= 1:
11     err <- call f2fs_get_dnode_of_data(ctx.dn, cluster <<
   log_cluster_size)
12     if err != 0:
13         return err
14     if data_blkaddr(ctx.dn) == COMPRESS_ADDR:
15         ctx.compressed_clusters <- 1
16         iomap.length <- min(length, cluster_end(index) - index)
17         iomap.type <- IOMAP_MAPPED
18     return 0

```

```

19 else:
20     all <- min(all, MAX_PBLKS_INFO_SIZE)
21     for i <- 0 to all - 1:
22         cidx <- cluster + i
23         if call f2fs_lookup_read_extent_cache(inode, cidx, ei):
24             ctx.pblks_arr[i] <- extent pblk info
25         else:
26             err <- call f2fs_get_dnode_of_data(ctx.dn, cidx <<
                log_cluster_size)
27             if err != 0:
28                 return err
29             if data_blkaddr(ctx.dn) != COMPRESS_ADDR:
30                 break
31             ctx.pblks_arr[i] <- dnode pblk list
32         process++
33     ctx.compressed_clusters <- process
34     iomap.length <- bytes(process clusters); iomap.type <-
        IOMAP_MAPPED
35     return 0
36 return call f2fs_buffered_read_iomap_begin(inode, pos, length, flags,
        iomap)

```

在旧的单簇路径中，iomap->length 被截断为从 pos 到当前簇末尾的长度——iomap 的 iomap 区间迭代器 外层循环每次迭代最多覆盖一个压缩簇的范围。新的多簇路径在检测到当前动态大页对齐于簇边界且跨越多个簇时（all_clusters > 1），遍历全部簇，从 dnode 获取每个簇的物理块地址信息，填入 物理块数组 数组，最后将 iomap->length 设置为覆盖所有连续压缩簇的字节总长度。这意味着外层 iomap 区间迭代器 一次迭代即可覆盖多个簇的物理范围，减少了迭代次数和映射查询开销。

内层迭代器：复用外壳，替换内胆。F2FS 压缩读页迭代器（算法 4-6）是本系统对 iomap 通用框架最关键的一处定制。

算法 4-6 f2fs_compress_readpage_iter: compressed read iterator

```

1 Input: iter, ctx
2 Output: advanced length or error
3 pos <- iter.pos
4 length <- iomap_length(iter)
5 folio <- ctx.cur_folio
6 fifs <- call f2fs_ifs_alloc(folio, iter.flags, false)
7 call iomap_adjust_read_range(iter.inode, folio, &pos, length, &poff,

```

```

    &plen)
8  if plen == 0:
9      goto done
10 if call iomap_block_needs_zeroing(iter, pos):
11     call zero_and_mark_uptodate(folio, poff, plen)
12     goto done
13 if ctx.compressed_clusters > 0:
14     ret <- call f2fs_do_read_multi_folios(ctx, pos, plen)
15 else:
16     ret <- call f2fs_do_read_single_folio_iomap(iter, ctx, pos, plen,
17         poff)
18 if ret != 0:
19     return ret
20 done:
21 length <- pos - iter.pos + plen
22 return call iomap_iter_advance(iter, &length)

```

该函数的结构与 `iomap` 的通用 `iomap` 读页迭代器同构——两者的外层迭代逻辑和边界处理保持一致，均依赖 `iomap` 调整读范围函数计算实际读取范围、`iomap` 块需填零判断函数处理空洞填零。差异仅在于内层的 I/O 提交策略：通用版本在确定需要读取的区间后直接分配块设备 I/O 请求、调用添加动态大页函数写入数据并立即提交；F2FS 压缩版本则根据压缩簇计数的值将子范围分发到两条路径——压缩路径调用 F2FS 多动态大页读函数将页面暂存到压缩上下文中延迟提交，等待簇满或预读结束时统一发起磁盘读取并解压；非压缩路径（压缩文件内的非压缩区域）调用 F2FS 单动态大页 `iomap` 读函数直接构建块设备 I/O 请求并提交。

这种"外壳复用、内胆替换"的策略，其必要性来源于压缩数据的延迟提交语义：`iomap` 读页迭代器的"立即块设备 I/O 请求提交"模式与压缩路径要求的"先聚合后解压"模式在根本上矛盾。直接修改通用 `iomap` 代码来迁就这一需求将影响所有其他文件系统，因此本系统选择在 F2FS 层实现一个结构同构但内部分发的迭代器。

完成计数：跨路径统一。两条 I/O 路径的完成回调最终均汇集到 F2FS 完成动态大页读函数。该函数每次被调用时从 F2FS 扩展型动态大页状态对象的未完成读字节计数中减去已完成子区间的字节数，仅当计数器归零（或回落至仅剩魔数偏置的 `F2FS_IFS_MAGIC`）时才调用动态大页读结束函数解锁动态大页。无论是解压回调还是块设备 I/O 请求完成回调先到达，在另一个子区间的处理完成之前计数器均不会归零。对于一个跨越 2 个压缩簇和 1 个非压缩区间的 64KB 动态大页，其未完成读字节

计数初始累加了全部子区间的字节总数，三条路径分别完成后各减去对应量，仅当全部完成时动态大页读结束函数才被调用。

4.4.2 压缩文件缓冲写适配

压缩文件写入中，以压缩簇为基本写入单位的方案存在"簇视角"与"动态大页视角"的根本分歧。一种直观方案是以压缩簇为基本写入单位：每次只处理一个簇大小的范围，动态大页的阶数受限于簇大小。这种方案下，一个 64KB 的写入请求若横跨 4 个簇（每个簇 4 个页，即 16KB），将被拆分为 4 次独立的小粒度操作——动态大页的聚合优势被完全抵消。

本系统提出一种"动态大页视角"的写入算法（算法 4-7）：一次性分配覆盖目标范围的动态大页，再按 head、middle、tail 三类簇区间处理旧数据读取和覆盖写入。图 4-5 展示了这一划分：head/tail 部分簇需要保留未覆盖数据，中间完整簇会被写入完全覆盖，可直接标记为就绪而跳过磁盘读。

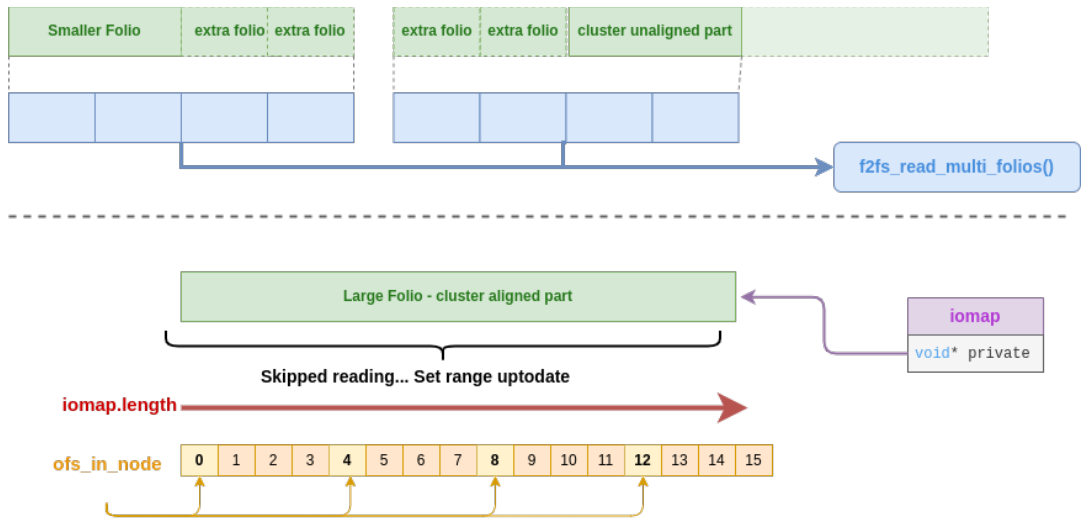


图 4-5 压缩文件缓冲写的 head/middle/tail 簇处理

算法 4-7 f2fs_buffered_write_iomap_begin: compressed range prepare

```
1 Input: inode, offset, length, flags
2 Output: iomap with prepared folio
3 folio <- call iomap_get_folio(iter, offset, length)
4 fifs <- call f2fs_ifs_alloc(folio, iter.flags, false)
5 if folio_order(folio) > 0 and fifs != NULL:
6     fifs.read_bytes_pending <- fifs.read_bytes_pending + 1
7 head <- first cluster touched by offset
8 tail <- last cluster covered by folio
9 if head is partial:
10     prepare head range; add pending bytes
```

```

11     if any page is stale:
12         call f2fs_read_multi_folios(cc, pblks, dn, bio)
13     call f2fs_reset_compress_ctx(cc)
14 for c <- first_full_cluster to last_full_cluster:
15     err <- call f2fs_ensure_dn_for_cluster(inode, c, dn)
16     if err != 0:
17         return err
18     if data_blkaddr(dn) != COMPRESS_ADDR:
19         goto normal_path_or_done
20     call mark_full_cluster_uptodate(folio, c)
21     iomap.length <- iomap.length + cluster_size
22 if tail is partial and tail != head:
23     prepare tail range; add pending bytes
24     if any page is stale:
25         call f2fs_read_multi_folios(cc, pblks, dn, bio)
26 if bio != NULL:
27     call f2fs_submit_read_bio(sbi, bio, DATA)
28 while fifs.read_bytes_pending != F2FS_IFS_MAGIC + 1:
29     call f2fs_io_schedule_timeout(DEFAULT_IO_TIMEOUT)
30 fifs.read_bytes_pending <- fifs.read_bytes_pending - 1
31 call folio_mark_uptodate(folio)
32 iomap.private <- folio; iomap.type <- IOMAP_MAPPED
33 return 0

```

该算法的关键机制是未完成读字节计数的"偏置-轮询"同步模式。在提交任何异步块设备 I/O 请求之前，先对未完成读字节计数加一个值为 1 的偏置——此偏置阻止任何块设备 I/O 请求完成回调触发动态大页读结束函数。随后 iomap 开始回调进入显式轮询循环，检查计数器是否降至 F2FS_IFS_MAGIC + 1；该值表示所有块设备 I/O 请求已完成，但偏置尚未移除。轮询退出后移除偏置，动态大页完全就绪，由其持有方控制锁的释放。

Read_bytes_pending 示意图

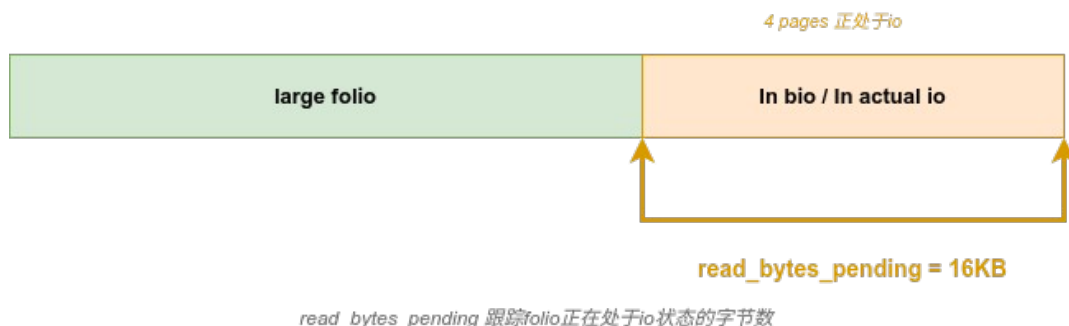


图 4-6 未完成读字节计数的偏置等待机制

算法的数据填充充分三个阶段。第一阶段处理 head 部分簇——写入起始位置不齐簇边界的首簇，通过 逐块 uptodate 位图检查该簇内各子页是否已就绪：若全部就绪则跳过读取，若有任何子页未就绪则通过 F2FS 多动态大页读函数 发起磁盘读取。第二阶段处理中间完整簇——这些簇将被整个写入覆盖，旧数据没有保留价值：直接调用 iomap 设置范围就绪函数 将对应动态大页子范围标记为 uptodate，完全跳过磁盘读取。第三阶段处理 tail 部分簇，逻辑与 head 簇相同。对于非压缩区域（data_blkaddr != COMPRESS_ADDR），算法回退到普通文件路径。

执行阶段，F2FS 获取动态大页函数检查 iomap->private 是否已被准备阶段设置——若是，直接返回已填充就绪的大动态大页；否则回退到通用 iomap 的 iomap 获取动态大页函数执行标准分配和加锁流程。这意味着 iomap 核心框架在写入数据时拿到的动态大页已经在 iomap 开始回调中完成了全部压缩数据的异步读取和同步等待。

旧方案将一个 64KB 写入拆为 4 次独立操作，动态大页阶数受限于簇大小。本系统的动态大页视角算法则一次分配覆盖全部簇的动态大页，在单次 iomap 开始回调中完成所有必要簇的数据准备。

4.5 动态大页脏页写回

动态大页条件下脏页写回存在两个失控层面。第一，清除动态大页脏标记函数 清除整个动态大页的 PG_dirty 标志后，写回路径失去了区分“哪些子块需要实际写回”的信息——一次 4KB 的局部修改可能迫使整个 64KB 动态大页进入日志追加写，产生 60KB 的无效写放大。第二，写回入口将动态大页拆解为页数组后，逐页调用同一个单页写回函数，该函数以动态大页起始索引作为块坐标查询节点树——对于起始索引为 0 的 64KB 动态大页，同一逻辑块被重复写入 16 次，而子块 1~15 的数据从未到达正确位置。

压缩文件的写回在此之上还叠加了簇级提交边界问题：一个动态大页跨越多个压缩簇时，每个簇对同一动态大页独立执行加锁/解锁循环，导致锁周期被反复打断——簇 A 的完成回调提前解锁动态大页，而动态大页的其他部分仍在等待簇 B 和簇 C 的压缩写出。

4.5.1 普通文件动态大页脏区间写回

本系统用 F2FS 单动态大页写回函数（算法 4-8）替代原有的逐页写回逻辑，在三个维度上进行控制。

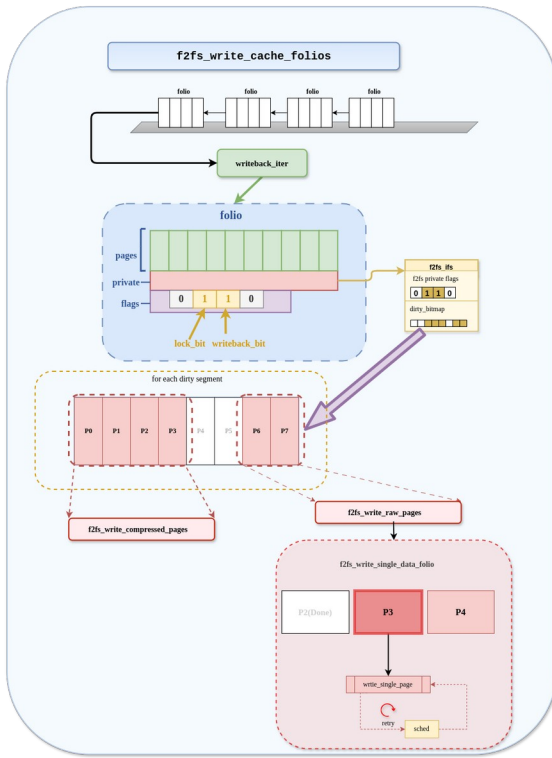


图 4-7 动态大页脏区间写回路径

算法 4-8 f2fs_write_single_data_folio: subrange writeback

```

1 Input: folio, wbc, compr_blocks, from_compress, start, end
2 Output: err, submitted_pages_count
3 if folio_order(folio) == 0:
4     err <- call f2fs_write_single_data_page(folio, wbc)
5     if err == -EAGAIN and sync writeback:
6         call f2fs_io_schedule_timeout(DEFAULT_IO_TIMEOUT)
7         goto retry_single
8     return err
9 fifs <- folio.private
10 first <- (start - folio_pos(folio)) >> PAGE_SHIFT
11 last <- (end - 1 - folio_pos(folio)) >> PAGE_SHIFT
12 for i <- first to last:
13     page <- folio_page(folio, i)
14     fio <- build_io_info(page, wbc, compr_blocks)
15     fio.need_lock <- compr_blocks ? LOCK_DONE : LOCK_RETRY
16 retry:
17     atomic_add(PAGE_SIZE, fifs.write_bytes_pending)
18     err <- call f2fs_do_write_data_page(fio)
19     if err == -EAGAIN and (sync writeback or from_compress):

```

```

20      call f2fs_io_schedule_timeout(DEFAULT_IO_TIMEOUT)
21      fio.need_lock <- LOCK_REQ
22      goto retry
23      if err != 0:
24          atomic_sub(PAGE_SIZE, fifs.write_bytes_pending)
25          break
26      submitted++
27      submitted_pages_count <- submitted
28      return err

```

子范围提交。函数接收字节粒度的 (start, end) 范围参数，传统写回函数隐含以整个动态大页为写入范围。循环的起止索引 folio_start_idx 和 folio_end_idx 由调用方传入的范围决定。上层的 F2FS 缓存动态大页写回函数通过 F2FS 查找脏范围函数从 F2FS 扩展型动态大页状态对象的 逐块 dirty 位图中定位动态大页内的实际脏区，仅将脏子范围传递给本函数。

脏区的判定不再依赖动态大页级的 PG_dirty 标志——该标志在清除动态大页脏标记函数被整体清除后，子块级别的脏信息保留在 逐块 dirty 位图中：由 4.2.3 节的 iomap 设置范围脏函数写入，由 F2FS 查找脏范围函数读取。对于一个仅有一个 4KB 子块变脏的 64KB 动态大页，(start, end) 仅覆盖该 4KB 范围，循环体只执行一次，写入量为 4KB——写放大从 16x 降至 1x。

完成生命周期管理。每个子块在提交前通过 atomic_add(PAGE_SIZE, &fifs->write_bytes_pending) 递增原子计数器。块设备 I/O 请求完成回调处理每个请求段时执行 atomic_sub_and_test(fi.length, &fifs->write_bytes_pending)：仅当计数器归零时才调用 folio_end_writeback(folio) 结束该动态大页的写回生命周期。这一机制确保了"一次块设备 I/O 请求完成"不再错误地等同于"一个动态大页完成"——多请求场景下，只有全部子块的块设备 I/O 请求均已完成，writeback 状态才被解除。

重试逻辑下沉。当 F2FS 数据页写函数因 checkpoint 全局锁冲突返回 -EAGAIN 时，传统写回路径需要在外层重新获取动态大页并从头重写整个范围。本系统将重试逻辑从外层下沉到子块级别——失败位置之后尚未提交的子块继续提交，已成功提交的子块不受影响。对于同步写回模式 (WB_SYNC_ALL) 和压缩文件写回，重试循环在等待超时后直接跳回当前子块的提交点 (goto retry_multi)，而非回退到动态大页起始索引。这避免了"64KB 动态大页中前 32KB 已提交成功、后 32KB 因 checkpoint 重试，却需将前 32KB 重新写入"的情况。

表 4-3 传统写回路径与子范围控制路径对比

指标	传统路径	本系统路径
----	------	-------

实际写入量	64KB（整动态大页）	4KB（仅脏子块）
写放大	16x	1x
日志消耗	追加 64KB	追加 4KB
写回完成判定	单次块设备 I/O 请求回调	未完成写回字节计数 归零
重试粒度	整动态大页	子块级别

4.5.2 压缩文件动态大页写回与延迟解锁

压缩文件的写回复用 F2FS 单动态大页写回函数 的核心循环，并在两个方向上叠加专属逻辑。

簇边界限制。F2FS 查找脏范围函数 在检测到压缩文件时，将 `range_end` 截断至当前压缩簇的末尾——`range_end = min(range_end, (cluster_i_end_idx(索引节点, *range_start >> PAGE_SHIFT) + 1) << PAGE_SHIFT)`。这意味着 F2FS 单动态大页写回函数 接收到的 `(start, end)` 永远不会跨越压缩簇边界——每次写回调最多处理一个簇内的子块。跨越多个簇的动态大页被拆分为按簇为单位的多次写回调，每次调用内部以子范围控制确保只有实际脏块被提交，簇的原子语义得到保持。

延迟解锁。压缩写回与普通写回的差异在于，脏数据不会在扫描到动态大页后立即形成最终块设备 I/O 请求，而是先按簇暂存到压缩上下文。此时动态大页的锁不能由单个子页写回函数或单个簇处理过程提前释放；解锁时机延后到该动态大页中进入压缩上下文的脏子范围全部被消费之后。解锁决策由未完成脏字节计数原子计数器驱动：压缩路径在提交整个簇的所有子块后，通过原子减并测试检查计数器是否归零，仅在归零时才调用动态大页解锁函数。

空悬动态大页收尾。F2FS 缓存动态大页写回函数在主循环结束后检查最后一个动态大页：若其仍处于锁定状态且没有任何子块被提交（`submitted == 0`，例如所有子块均为干净页），则手动调用动态大页解锁函数和动态大页写回结束函数，防止其永久空悬。

4.6 垃圾回收路径的动态大页适配

GC 搬移路径中，F2FS 数据页写函数 以动态大页的起始页缓存索引作为块坐标来查询节点树——当 GC 请求搬移的逻辑块落在一个动态大页的覆盖范围之内时（例如起始索引为 0 的 64KB 动态大页，GC 要搬移逻辑块号 5），写回函数以索引 0 而非 5 定位节点树条目，将错误块的旧数据写到新段。

本系统在 GC 路径的两个关键点上进行修正，使其与前述所有机制保持一致。

子范围脏区标记。GC 后台搬移路径（BG_GC）中的数据页处理函数在标记搬移目标页为脏时，根据动态大页阶数做出分支（算法 4-9）。

算法 4-9 `move_data_block`: GC subrange migration

```

1 Input: inode, bidx, gc_type
2 Output: err
3 folio <- call f2fs_get_lock_data_folio(inode, bidx, true)
4 if folio is writeback and gc_type == BG_GC:
5     return -EAGAIN
6 call f2fs_set_folio_private_gcing(folio)
7 if gc_type == BG_GC:
8     if folio_order(folio) == 0:
9         call folio_mark_dirty(folio)
10    else:
11        off <- (bidx - folio.index) << PAGE_SHIFT
12        call iomap_set_range_dirty(folio, off, PAGE_SIZE)
13    call f2fs_folio_put(folio, true)
14    return 0
15 call folio_mark_dirty(folio)
16 err <- call f2fs_do_write_data_page(fio)
17 if err == -EAGAIN:
18     goto retry_fg
19 call f2fs_folio_put(folio, true)
20 return err

```

对 0 阶动态大页，沿用原有的全动态大页脏标记（动态大页标记脏函数）；对高阶动态大页，调用 `iomap` 设置范围脏函数 以 `bidx`（GC 选中的目标逻辑块号）为偏移，仅标记动态大页内该子块对应的 4KB 范围为脏。这意味着 4.5 节的 F2FS 单动态大页写回函数写回路径读取 逐块 `dirty` 位图后仅提交该 4KB 范围——GC 搬移一个 4KB 子块不再触发整个 64KB 动态大页的写回。

同时，F2FS 设置 GC 标记函数通过 4.3 节 F2FS 扩展型动态大页状态对象的统一 API 将 `ONGOING_MIGRATION` 标志写入柔性数组的私有标志区，而非直接编码在动态大页的 `private` 字段 中，消除了与 `iomap` 指针的语义冲突。

块设备 I/O 请求子页偏移修正。 GC 读路径的 F2FS 提交页读函数在向块设备 I/O 请求添加页面时，以子页在动态大页内的相对索引计算偏移—— $(\text{index} - \text{folio} \rightarrow \text{index}) \ll \text{PAGE_SHIFT}$ ——确保块设备 I/O 请求只针对正确的子页范围操作。GC 搬移写路径的 F2FS 提交页块设备 I/O 请求函数同样使用 `fio->idx` 和 `fio->cnt` 精确指定子范围。这两处修正使 GC 路径的块设备 I/O 请求偏移不再依赖"一个动态大页一个块"的隐含假设。

4.7 本章小结

本章给出了 F2FS 动态大页缓冲 I/O 的整体设计方案，各项设计与其所解决问题的对应关系如表 4-4 所示。

表 4-4 问题与方案对应

问题	方案	核心机制
全路径"一个对象=一个块"假设	4.2.1 F2FS 区间设置 统一转换层	块号→字节区间，空洞/NEW_ADDR 精确区分
读路径单块映射（16 次调用）	4.2.2 区间读 iomap 开始回调 + 连续映射合并	映射调用 16→1，锁次数同步下降
写路径逐块分配	4.2.3 区间预分配 + NEW_ADDR 合并	元数据操作 N→1，批量空间预留
folio->private 所有权冲突	4.3 F2FS 扩展型状态对象 扩展型状态对象	头部兼容+柔性数组扩展+魔数类型识别+强制分配
压缩读混合 I/O 完成时机不一	4.4.1 多簇 iomap 开始回调 + 复刻迭代器 + 未完成读字节计数	解压与直接读在统一计数器下收敛
压缩写"簇视角"限制动态大页阶数	4.4.2 动态大页视角算法 + 跳过全覆盖簇	动态大页阶数不再受限于簇大小
写回脏区粗化 + 索引错位	4.5.1 逐块 dirty 位图 + (start,end) 范围 + 未完成写回字节计数	写放大 16x→1x，重试粒度下沉到子块
压缩写回跨簇锁冲突	4.5.2 簇边界限制 + 延迟解锁 延迟解锁	簇语义保持，锁由计数器统一驱动
GC 搬移子块偏移错误	4.6 子范围脏标记 +块设备 I/O 请求偏移修正	GC 与前台路径共享同一状态框架

这些设计共享一套底层基础设施：F2FS 区间设置函数提供块到区间的统一转换，F2FS 扩展型动态大页状态对象 提供跨路径的统一状态管理，逐块位图提供子块级别的脏区和 uptodate 判定。在此基础上，普通文件路径、压缩文件路径、写回路径和 GC 路径各自叠加专属逻辑，但均不脱离这套基础设施——避免了各路径各自形成独立适配方案而导致的维护分叉和语义不一致。

第五章 实验设计与性能评估

5.1 测试环境与配置

为验证本文方案在实际运行环境中的效果，实验分别部署在 QEMU 虚拟机与树莓派 5 两款平台上。QEMU 环境用于在受控条件下对比原生 F2FS 与引入动态大页及

iomap 适配后的改造内核在顺序 I/O 场景下的表现；树莓派 5 环境则用于检验同一套改动在真实 ARM 处理器和外接 SSD 上的行为是否与虚拟化环境一致。

下文两张表分别汇总了两套测试平台的关键配置参数。

表 5-1 QEMU 虚拟机测试环境配置

参数	配置
虚拟化平台	QEMU (aarch64)
内存	4 GB
存储设备	virtio-blk 虚拟磁盘
文件系统	F2FS，挂载选项：defaults,noatime
内核基线	Linux 6.15，原生 F2FS
改造内核	Linux 6.15，引入动态大页+ iomap 适配补丁
测试工具	fio

表 5-2 树莓派 5 测试环境配置

参数	配置
硬件平台	Raspberry Pi 5 (ARM Cortex-A76)
内存	8 GB
存储设备	外接 SATA SSD (通过 USB 3.0)
文件系统	F2FS，挂载选项：defaults,noatime
内核基线	Linux 6.15，原生 F2FS
改造内核	Linux 6.15，引入动态大页+ iomap 适配补丁
测试工具	fio

5.2 测试方法与负载设计

5.2.1 工作负载

实验以顺序读写（sequential read/write）为核心负载，使用 fio 工具生成单线程顺序访问请求。选择顺序读写作为主要评价负载的原因在于：本文的核心优化路径——动态大页与 iomap 区间映射——在连续文件访问场景下收益最为显著，页缓存对象粒度的扩大和映射次数的减少直接体现在大块连续 I/O 的吞吐能力上。

每次测试使用三类文件：普通文件、稀疏文件（sparse file）和压缩文件（compressed file），以覆盖本文实现所涉及的三种主要文件类型。测试文件大小均设置为大于系统可用内存，确保冷读（cold read）场景下每次读取均需从存储设备实际获取数据，避免页缓存命中掩盖路径开销差异。

5.2.2 测试矩阵

实验围绕以下维度的组合展开：

表 5-3 测试矩阵

维度	取值	说明
文件类型	普通文件、稀疏文件、压缩文件	覆盖普通映射、空洞映射和压缩簇路径
请求块大小	4KB、16KB、64KB、128KB、256KB、512KB、1MB	覆盖从小粒度到大粒度连续访问的完整范围
读写模式	顺序写、冷顺序读、热顺序读	冷读前清理页缓存，热读紧随首轮读取执行

5.2.3 评价指标

实验主要采集以下指标：

表 5-4 评价指标

指标	单位或形式	用途
吞吐量	MiB/s	反映文件系统顺序 I/O 路径的数据传输能力
IOPS	ops/s	辅助衡量小粒度请求下的路径开销
CPU 利用率	用户态与内核态占比	评估改造路径是否引入额外 CPU 开销

5.3 实验结果与分析

5.3.1 QEMU 虚拟机平台

图 5-1 至图 5-3 展示了 QEMU 平台上原生 F2FS 与改造后 F2FS 在普通文件、稀疏文件和压缩文件三类场景下的顺序 I/O 带宽对比。

Bandwidth comparison - normal file - QEMU VM

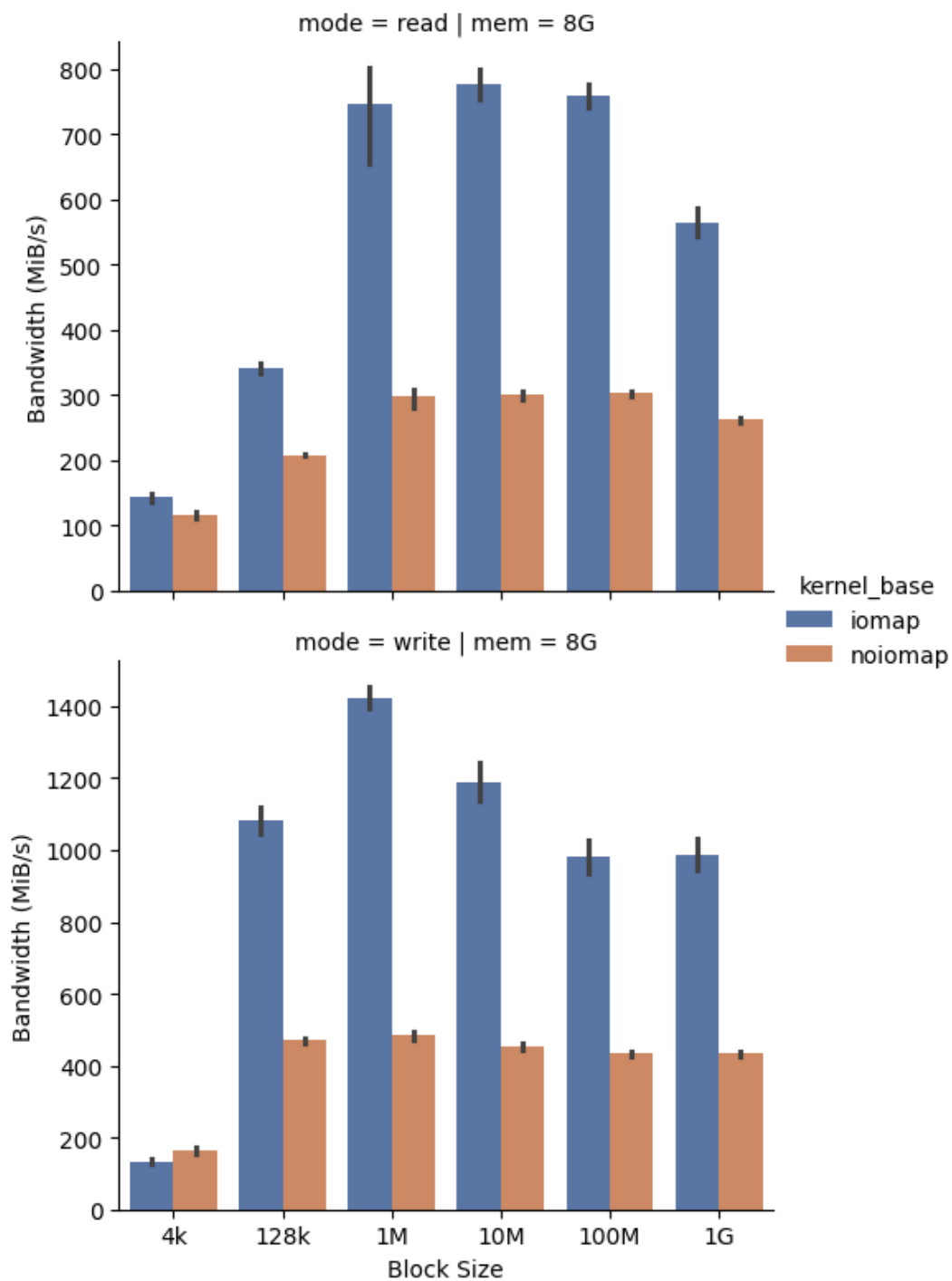


图 5-1 QEMU 平台普通文件顺序 I/O 带宽对比

Bandwidth comparison - sparse file (hole) - QEMU VM

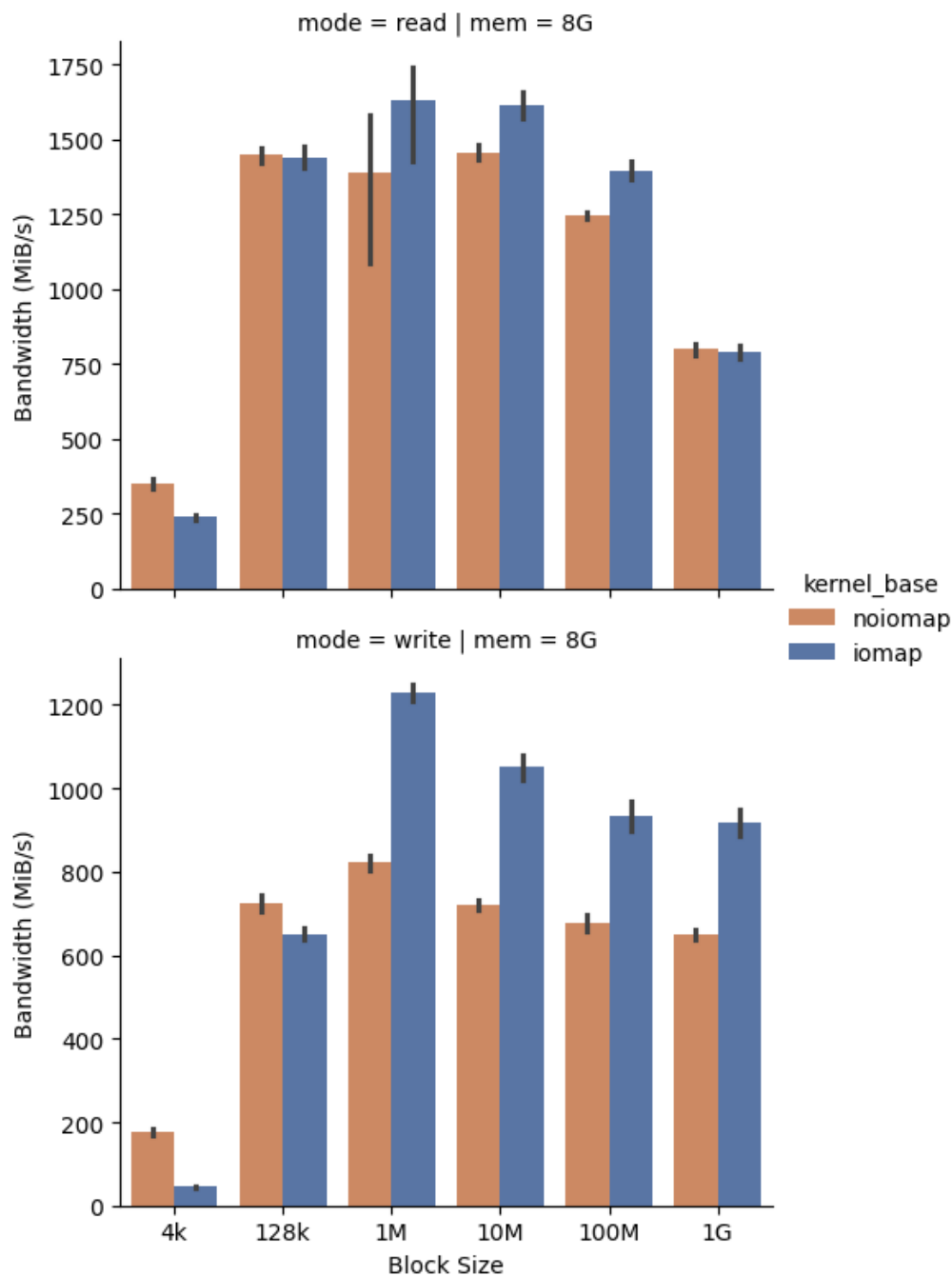


图 5-2 QEMU 平台稀疏文件顺序 I/O 带宽对比

Bandwidth comparison - compressed file - QEMU VM

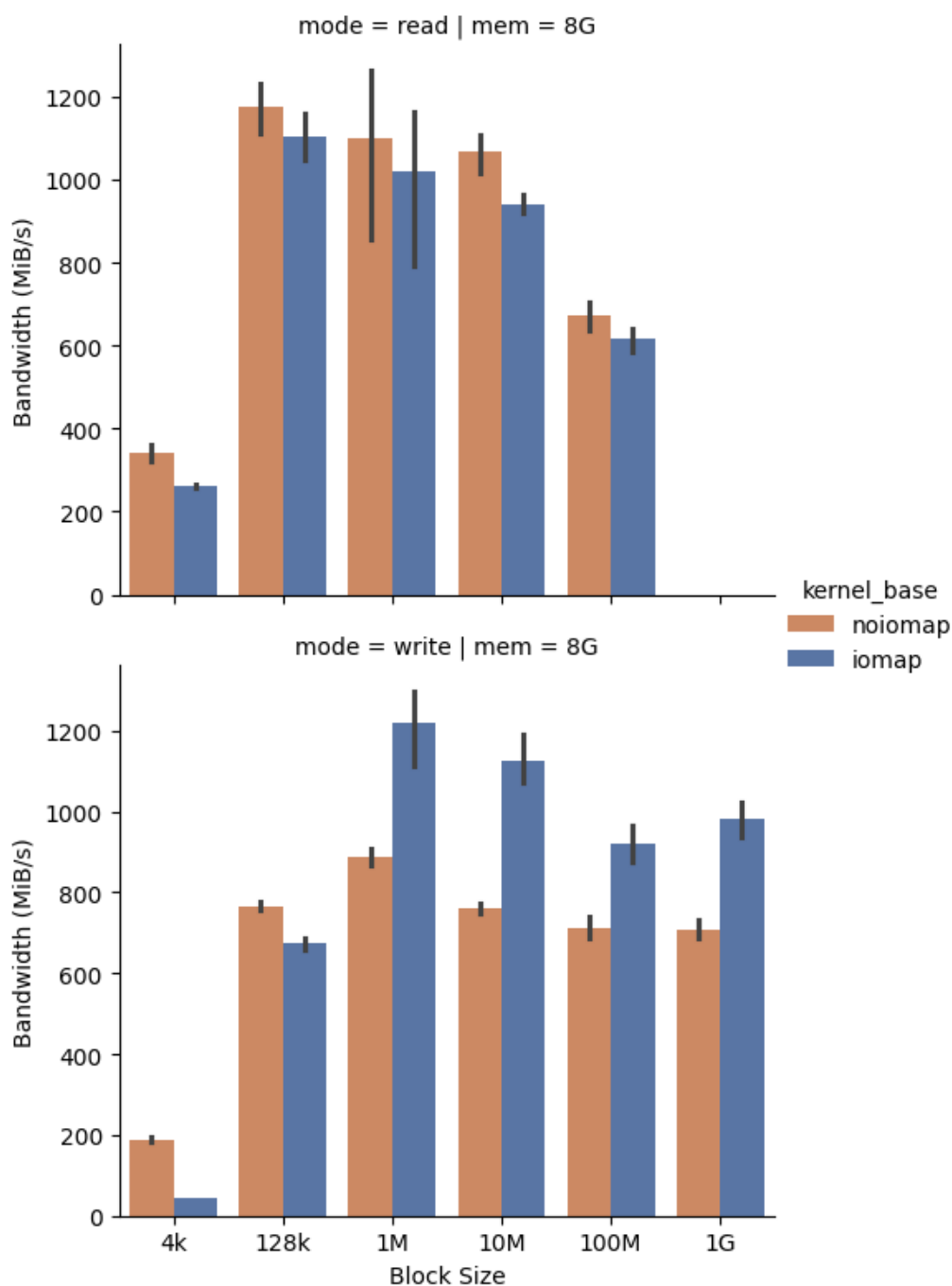


图 5-3 QEMU 平台压缩文件顺序 I/O 带宽对比

总体来看，改造后路径在普通文件的大块顺序读写场景中提升最明显，在稀疏文件和压缩文件写入场景中也能获得稳定收益。

在普通文件场景中，改造后路径的带宽提升最为直接。顺序读场景下，改造后路径在 128KB 请求大小时约为原生路径的 1.6 倍，在 1MB 和 10MB 请求大小下分别约为 2.5 倍和 2.6 倍，在 1GB 请求大小下仍保持约 2.2 倍提升。顺序写场景下，除 4KB 小块写入外，改造后路径均明显高于原生路径：128KB 请求大小下约为 2.3 倍，

1MB 请求大小下达到约 2.9 倍，10MB 请求大小下约为 2.6 倍，100MB 和 1GB 请求大小下约为 2.3 倍。

在稀疏文件场景中，改造后路径在读场景中的收益相对有限，整体表现为与原生路径接近：1MB 至 100MB 请求大小下约为原生路径的 1.1 至 1.2 倍，4KB 和 1GB 请求大小下则基本持平或略低。顺序写场景中，改造后路径在大块请求下体现出更稳定的提升，其中 1MB、10MB、100MB 和 1GB 请求大小下分别约为原生路径的 1.5 倍、1.5 倍、1.4 倍和 1.4 倍。

在压缩文件场景中，顺序读带宽整体与原生路径接近，部分块大小下略低于原生路径，说明压缩文件读路径的性能不仅受页缓存对象粒度影响，还受到压缩簇读取、解压和数据填充等因素制约。相比之下，压缩文件顺序写在大块请求下收益更明显：1MB 请求大小下约为原生路径的 1.4 倍，10MB 请求大小下约为 1.5 倍，100MB 和 1GB 请求大小下约为 1.3 至 1.4 倍。

5.3.2 树莓派 5 平台

图 5-4 至图 5-6 展示了树莓派 5 平台上相同负载的带宽对比。

Bandwidth comparison - normal file - Raspberry Pi 5

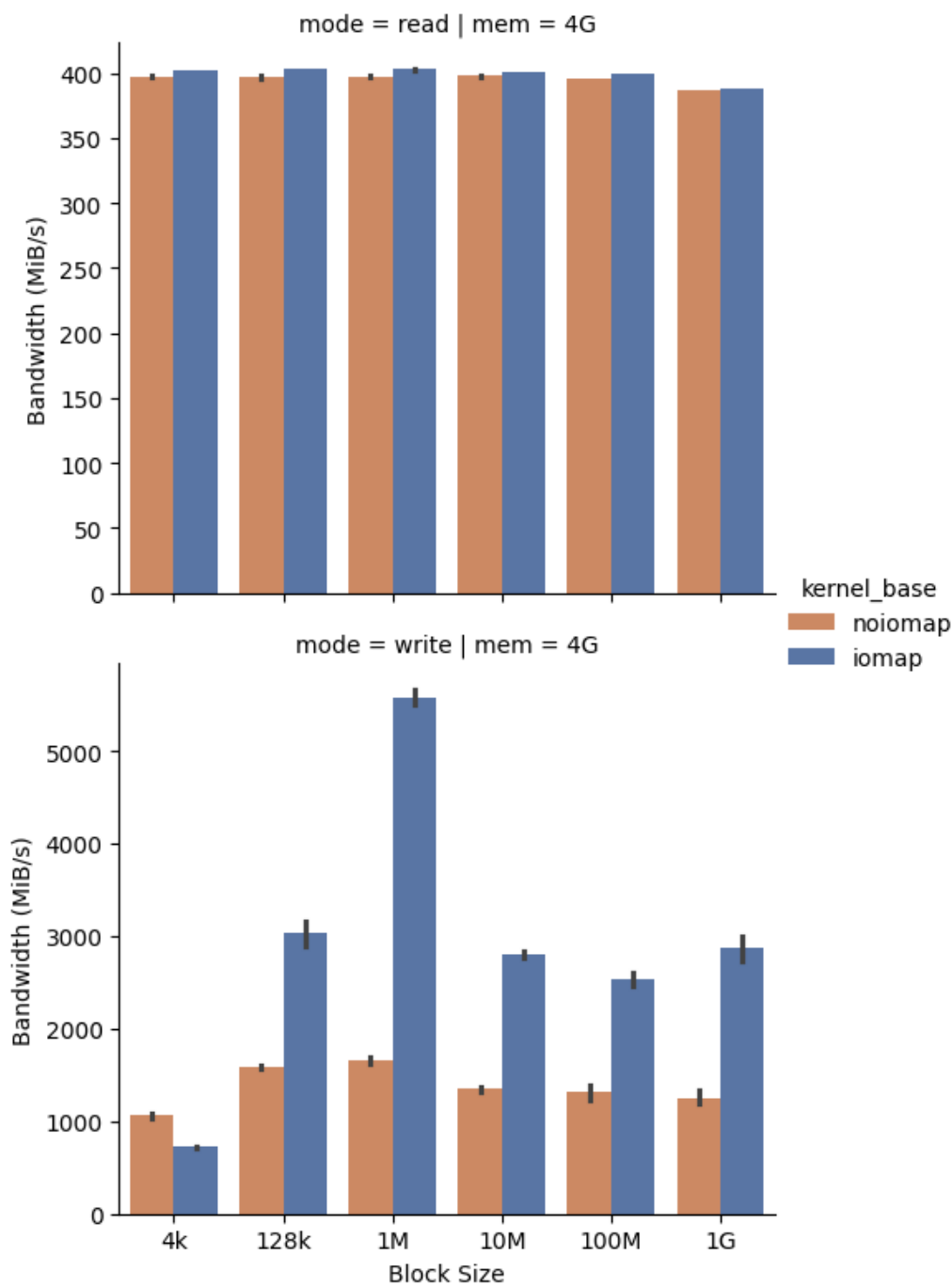


图 5-4 树莓派 5 平台普通文件顺序 I/O 带宽对比

Bandwidth comparison - sparse file (hole) - Raspberry Pi 5

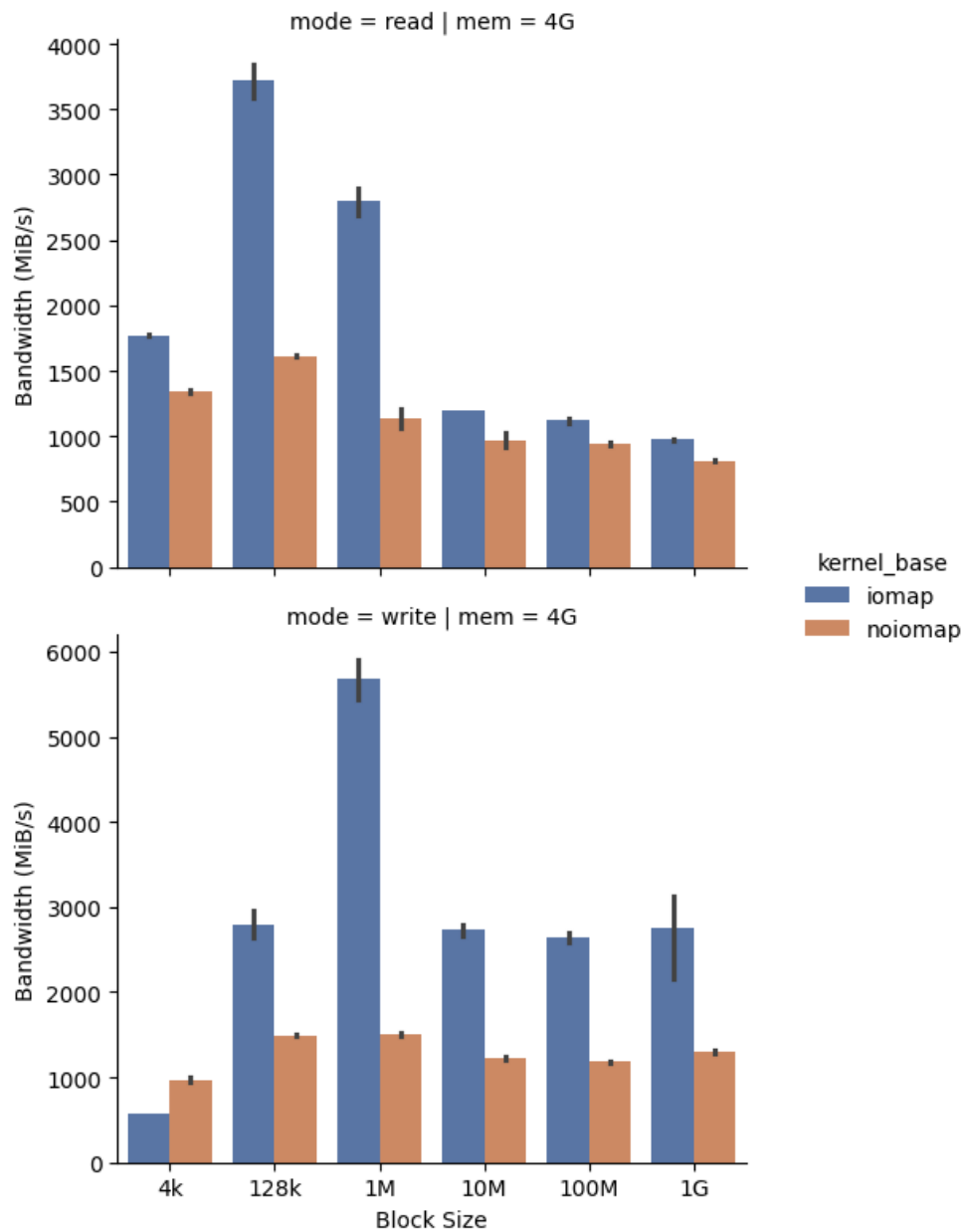


图 5-5 树莓派 5 平台稀疏文件顺序 I/O 带宽对比

Bandwidth comparison – compressed file – Raspberry Pi 5

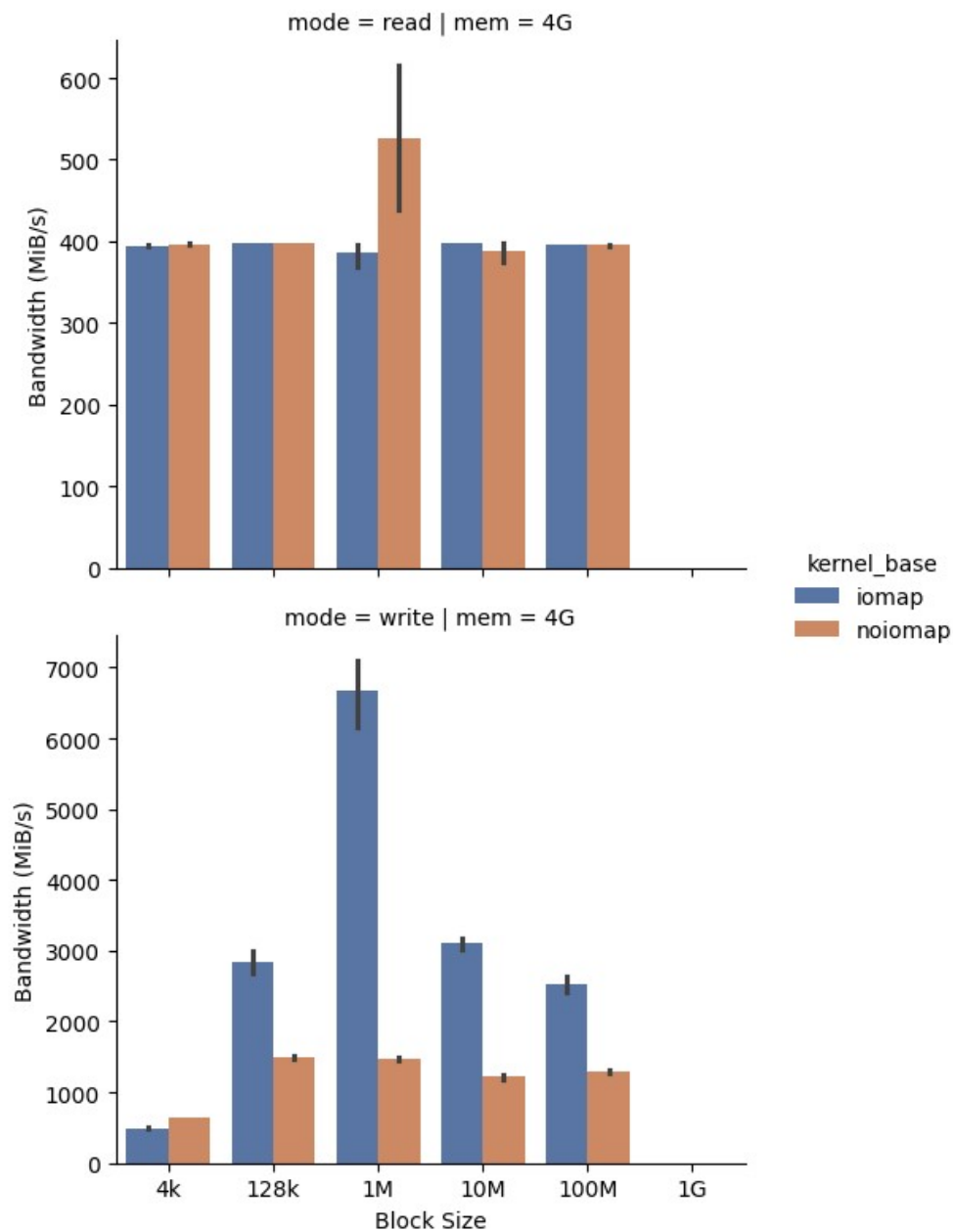


图 5-6 树莓派 5 平台压缩文件顺序 I/O 带宽对比

与 QEMU 环境相比，树莓派平台受 USB 3.0 接口、外接 SSD 性能和 ARM 处理器执行能力共同影响，带宽绝对值和波动特征有所不同，但改造后路径在大块顺序写场景中的提升趋势更加明显。

在普通文件场景中，顺序读带宽在改造前后基本持平，改造后约为原生路径的 1.0 倍，说明真实设备上的普通文件读路径主要受外接 SSD 读带宽和预读策略限制。顺序写场景则表现出明显提升：128KB 请求大小下约为原生路径的 1.9 倍，1MB 请求大小下达到约 3.4 倍，10MB 和 100MB 请求大小下约为 2.1 倍和 1.9 倍，1GB 请求

大小下约为 2.3 倍。该结果表明，动态大页对缓冲写路径中页缓存对象组织和脏页写回准备过程的影响，在真实 ARM 平台上同样能够转化为吞吐提升。

在稀疏文件场景中，改造后路径在读写两类负载下均有较明显收益。顺序读场景下，4KB 请求大小约为原生路径的 1.3 倍，128KB 和 1MB 请求大小下分别达到约 2.3 倍和 2.5 倍，10MB 及以上请求大小下仍保持约 1.2 倍左右提升。顺序写场景下，128KB 请求大小约为原生路径的 1.9 倍，1MB 请求大小下达到约 3.8 倍，10MB、100MB 和 1GB 请求大小下约为 2.1 至 2.3 倍。该结果说明，在真实设备上，稀疏文件路径中的空洞处理并没有阻断动态大页和 iomap 对大块 I/O 组织的收益。

在压缩文件场景中，顺序读整体与原生路径接近，多数块大小下约为原生路径的 1.0 倍；其中 1MB 请求大小下改造后读带宽低于原生路径，约为原生路径的 0.7 倍，表明压缩读路径仍可能受到压缩簇粒度、解压开销和缓存填充方式的影响。顺序写场景中，改造后路径提升显著：128KB 请求大小下约为原生路径的 1.9 倍，1MB 请求大小下达到约 4.5 倍，10MB 请求大小下约为 2.6 倍，100MB 请求大小下约为 2.0 倍。该结果表明，本文对压缩写路径的扩展能够在保持压缩簇语义的同时，将动态大页应用到大块连续写入过程，并在真实平台上获得最高约 4.5 倍的写入吞吐提升。

Bandwidth stability (box, 10 runs) - normal file - QEMU VM

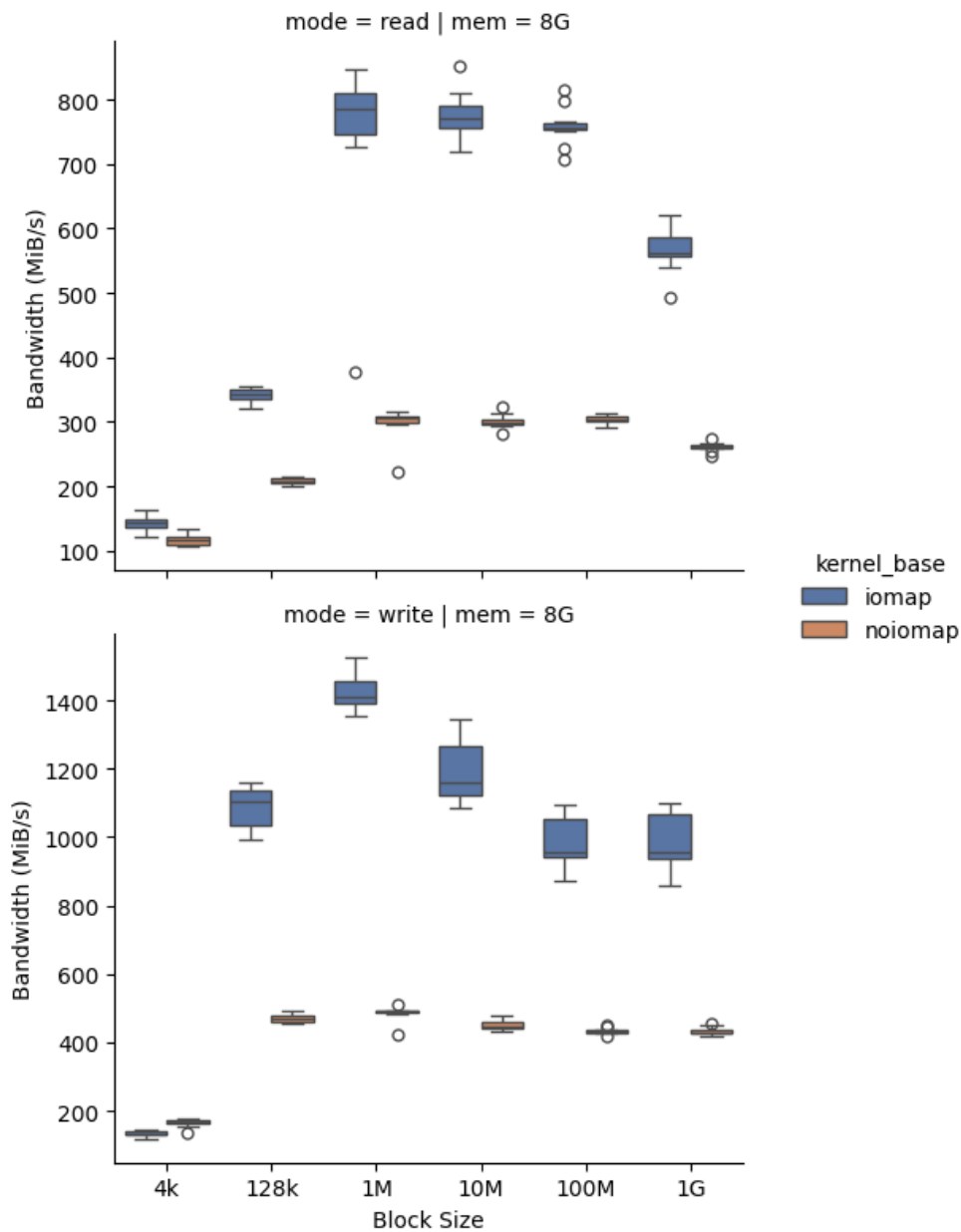


图 5-7 QEMU 平台普通文件带宽稳定性对比

Bandwidth stability (box, 10 runs) - sparse file (hole) - QEMU VM

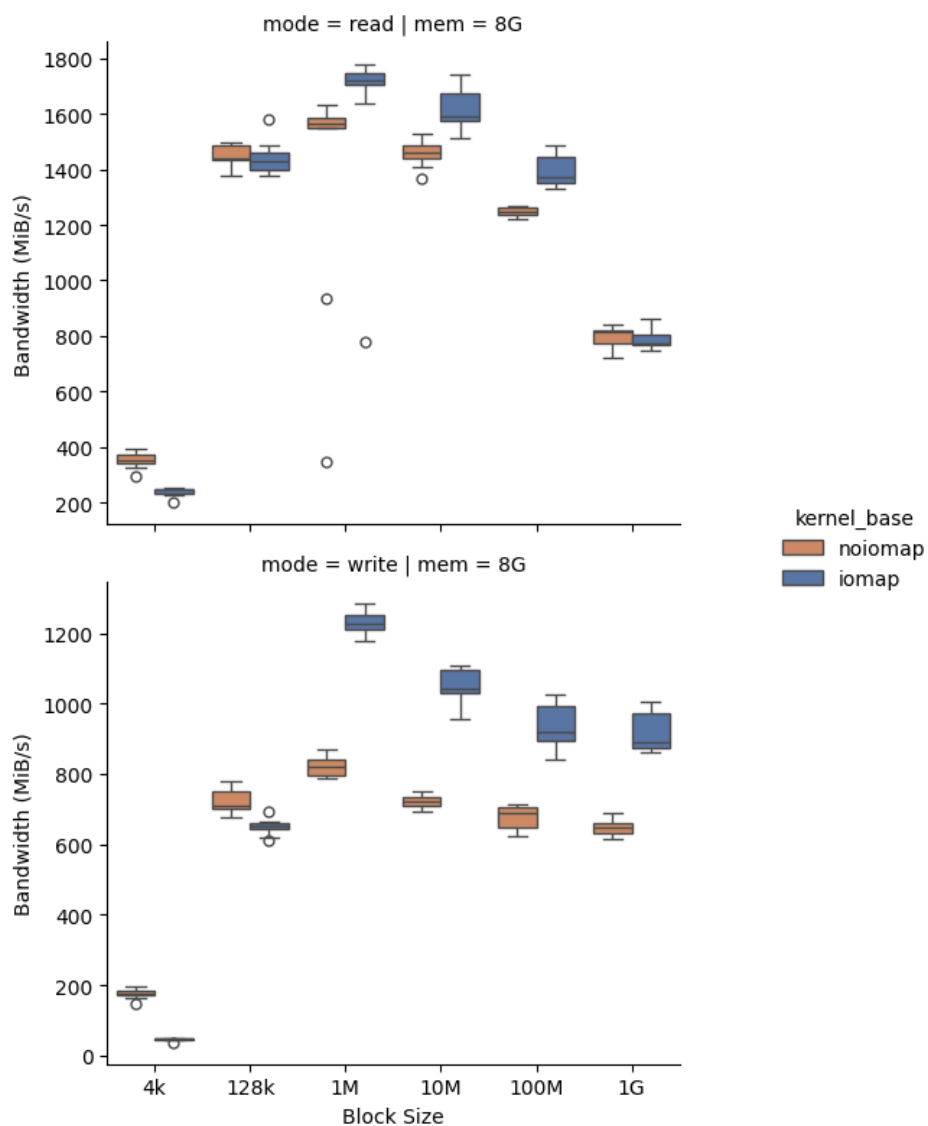


图 5-8 QEMU 平台稀疏文件带宽稳定性对比

Bandwidth stability (box, 10 runs) - compressed file - QEMU VM

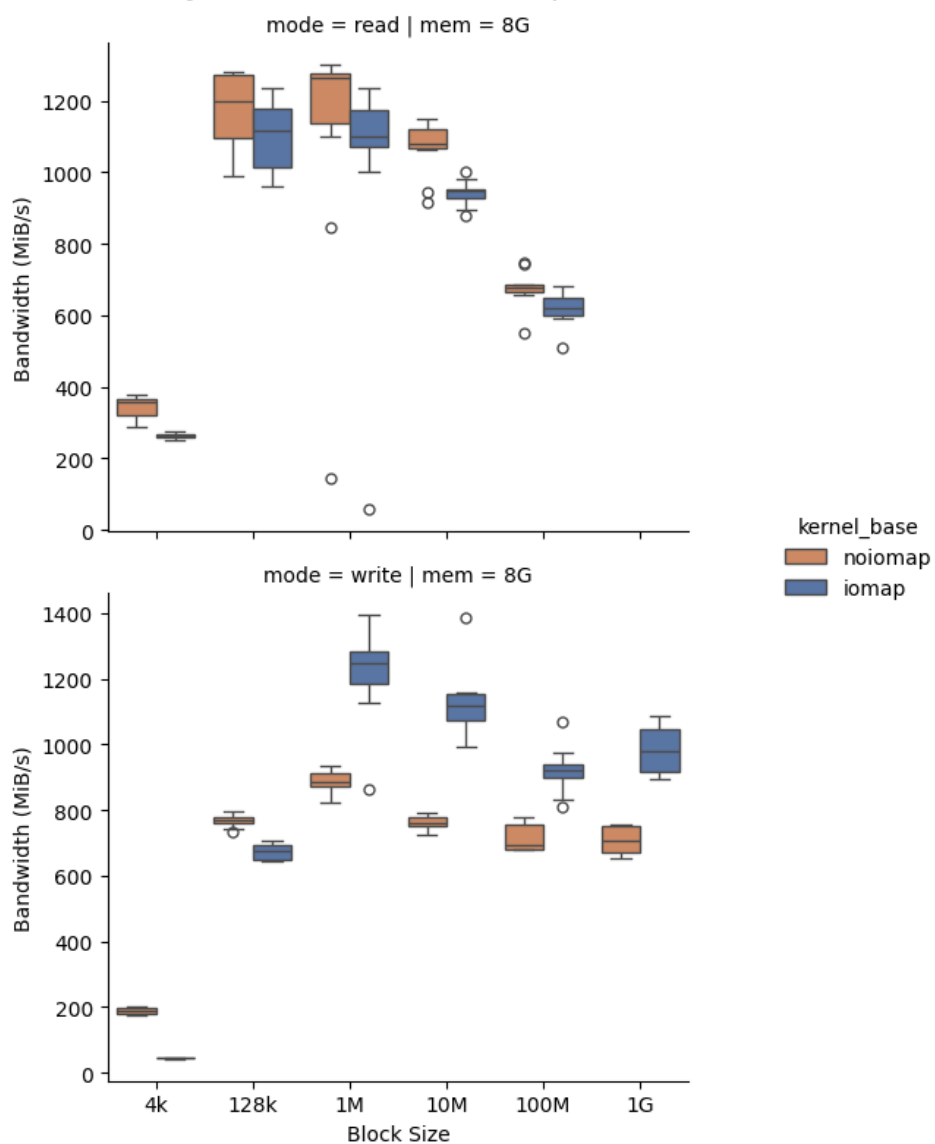


图 5-9 QEMU 平台压缩文件带宽稳定性对比

Bandwidth stability (box, 10 runs) - normal file - Raspberry Pi 5

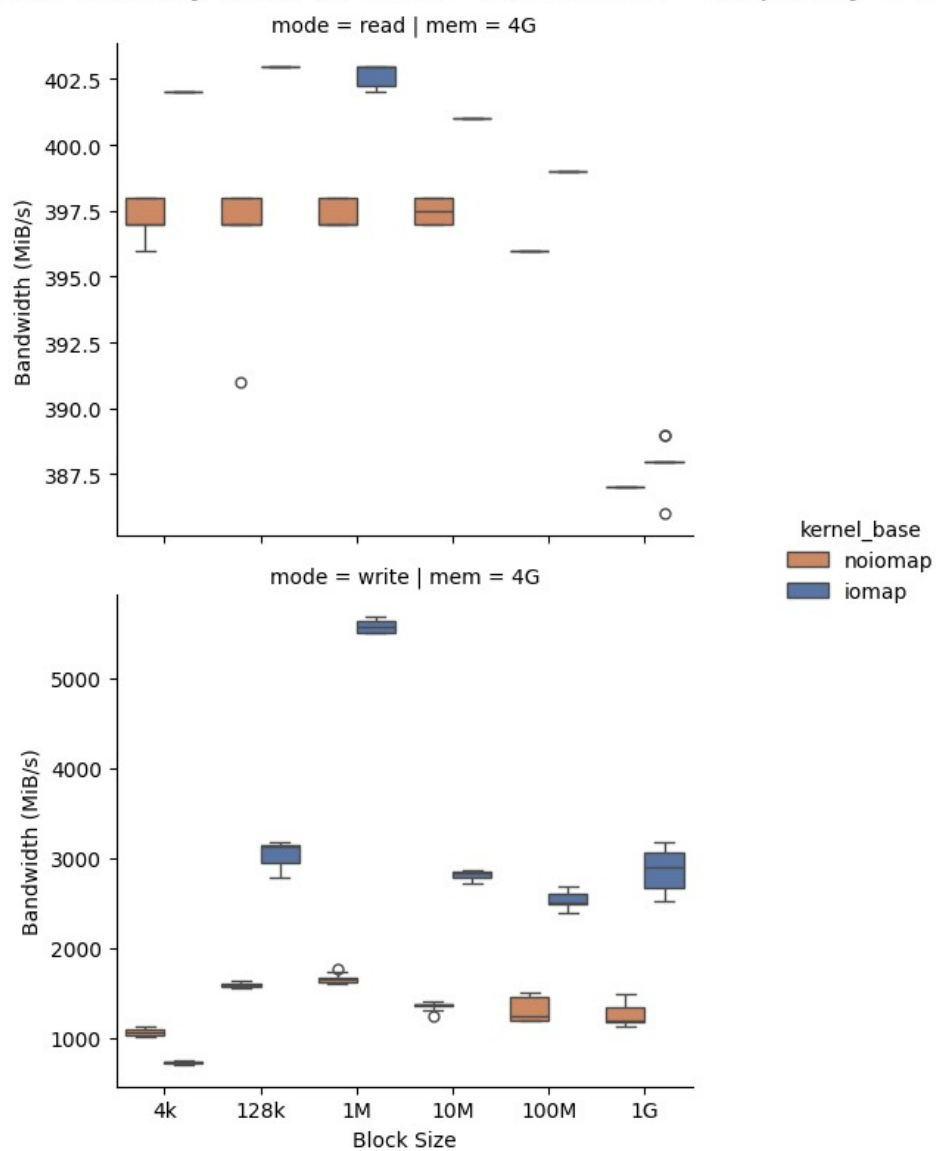


图 5-10 树莓派 5 平台普通文件带宽稳定性对比

Bandwidth stability (box, 10 runs) – sparse file (hole) – Raspberry Pi 5

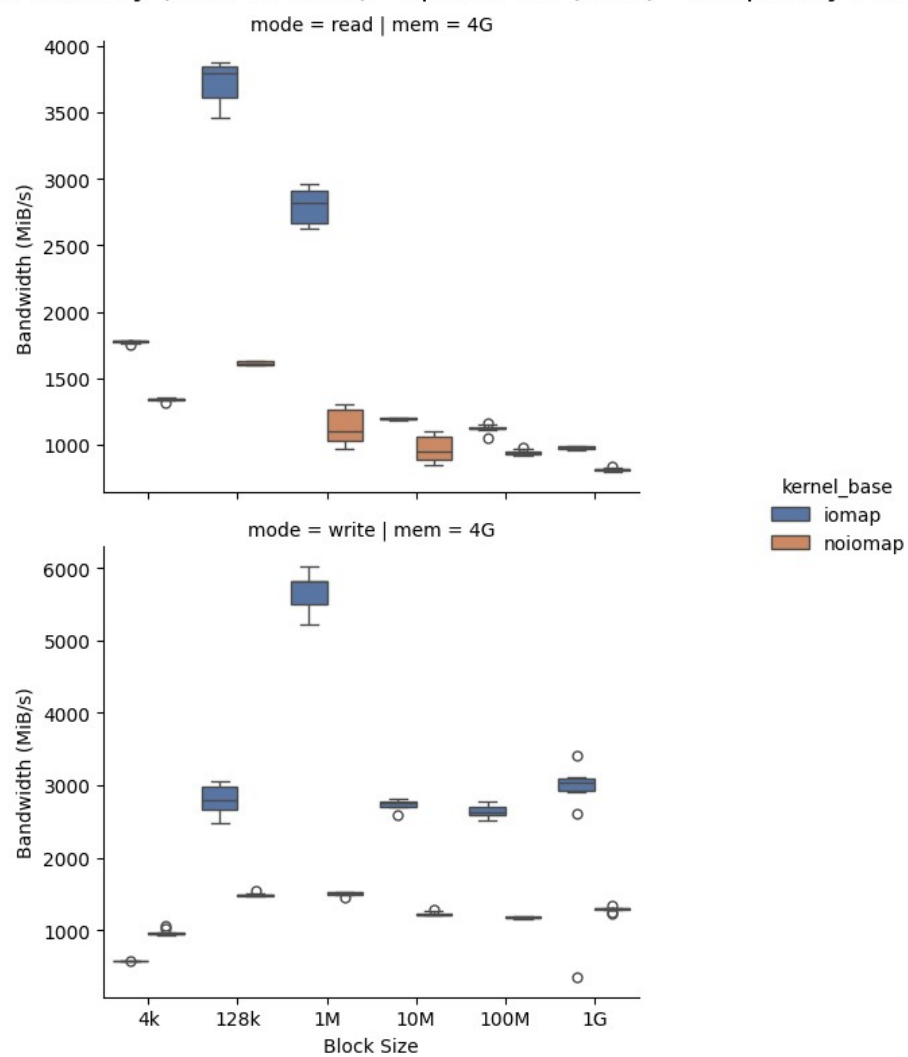


图 5-11 树莓派 5 平台稀疏文件带宽稳定性对比

Bandwidth stability (box, 10 runs) – compressed file – Raspberry Pi 5

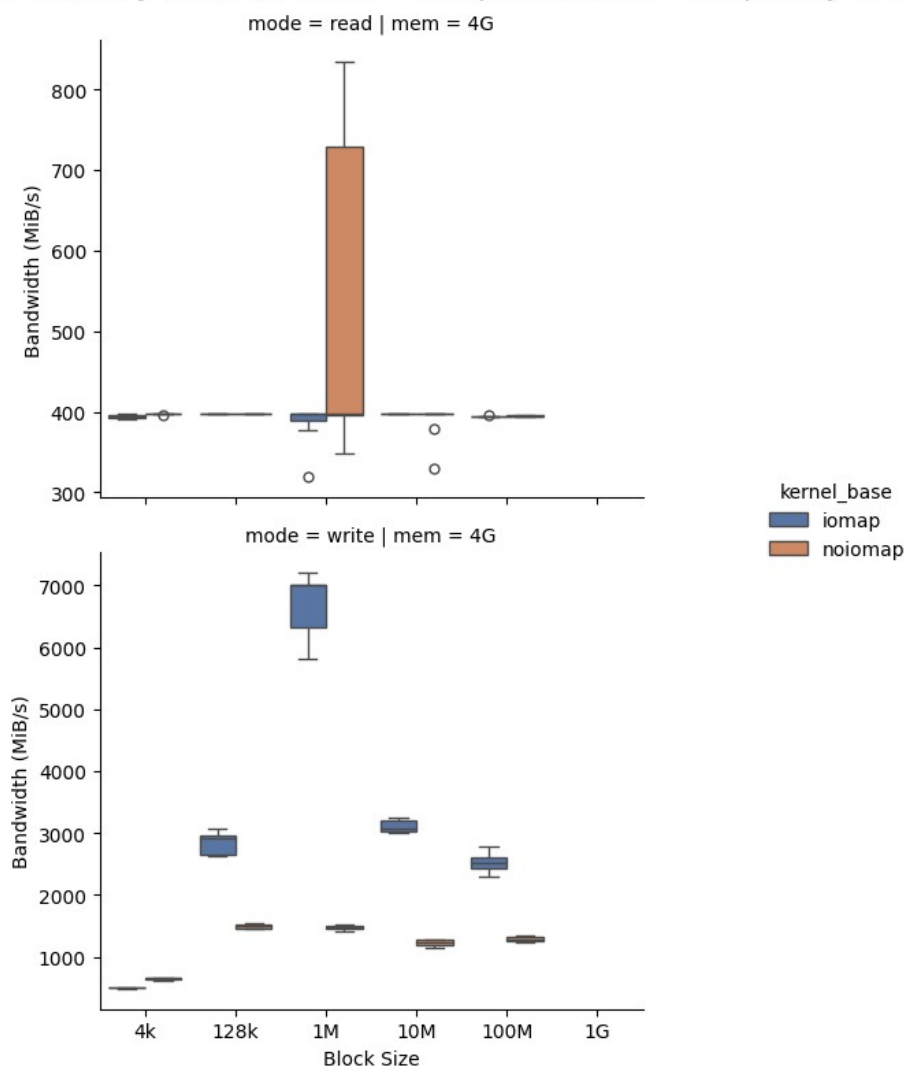


图 5-12 树莓派 5 平台压缩文件带宽稳定性对比

5.3.3 CPU 利用率分析

除带宽外，实验还采集了顺序读写过程中的 CPU 利用率。整体来看，改造后路径没有因为引入动态大页和 iomap 而带来额外的 CPU 占用放大。在 QEMU 普通文件大块负载中，原生路径的内核态 CPU 占比在多个块大小下接近或超过 90%，而改造后路径大多位于约 75% 至 80% 区间；在树莓派普通文件大块负载中，原生路径的内核态 CPU 占比约为 60%，改造后路径多处降至约 50% 左右。该趋势与本文的设计目标一致：动态大页减少页缓存对象数量，iomap 以区间迭代替代逐块状态遍历，从而降低了部分小粒度路径管理开销。

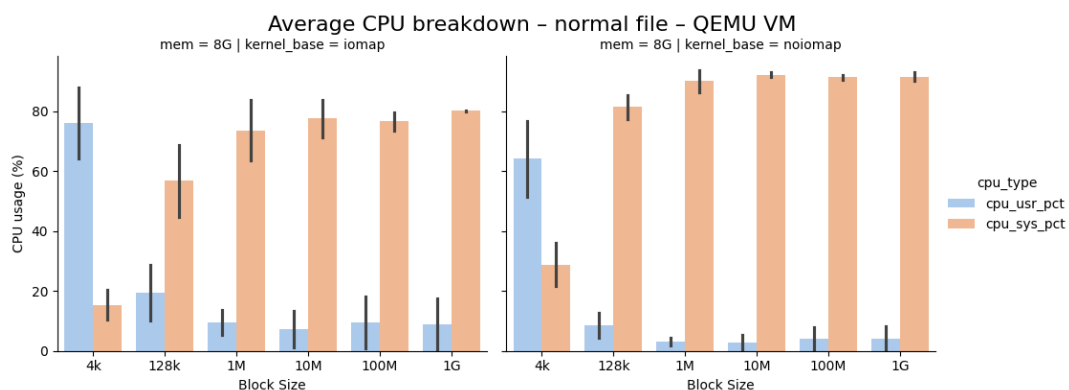


图 5-13 QEMU 平台普通文件 CPU 利用率对比

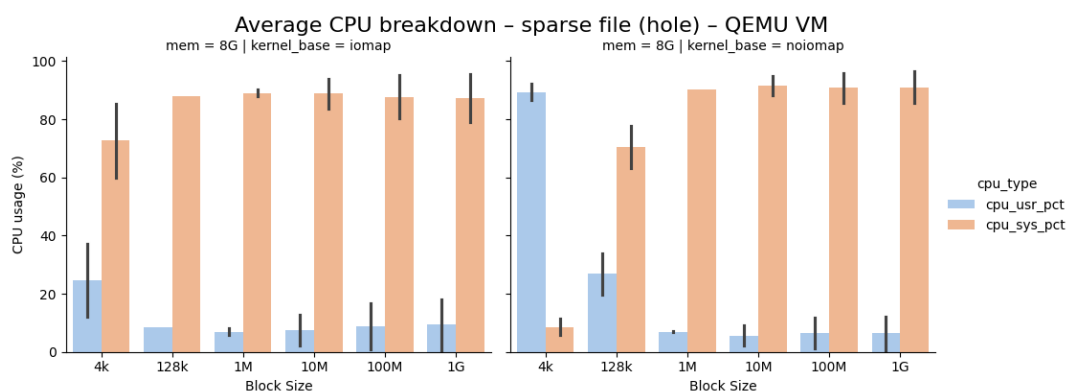


图 5-14 QEMU 平台稀疏文件 CPU 利用率对比

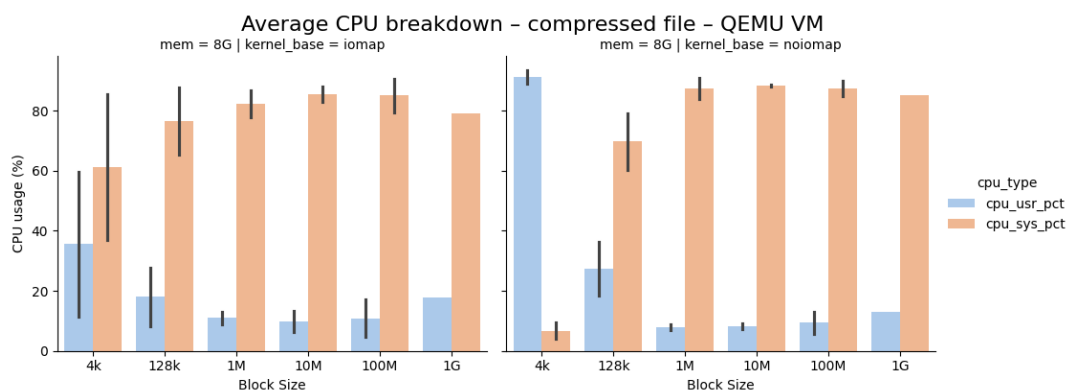


图 5-15 QEMU 平台压缩文件 CPU 利用率对比

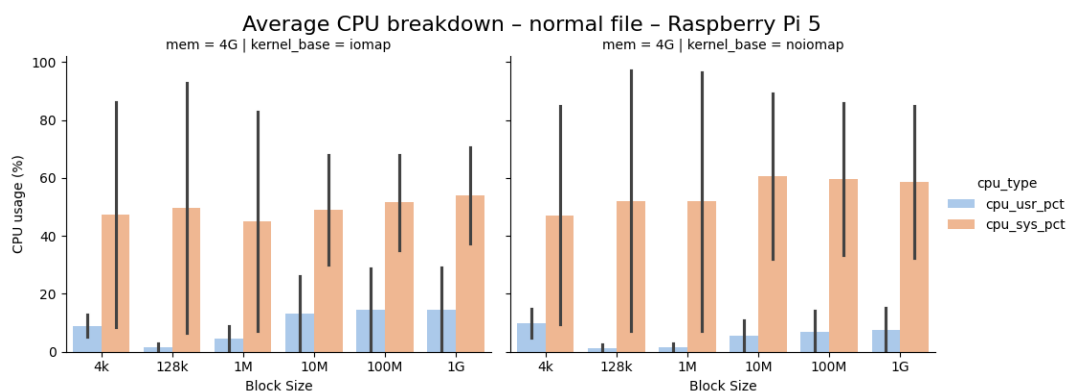


图 5-16 树莓派 5 平台普通文件 CPU 利用率对比

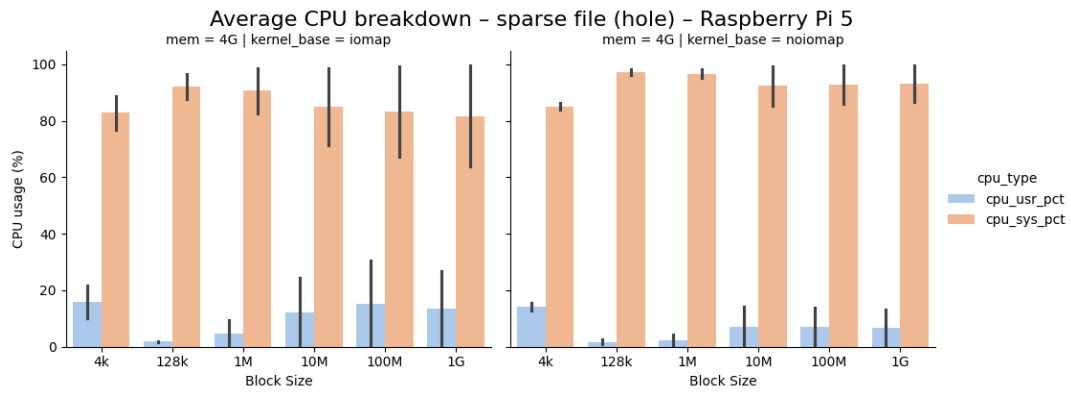


图 5-17 树莓派 5 平台稀疏文件 CPU 利用率对比

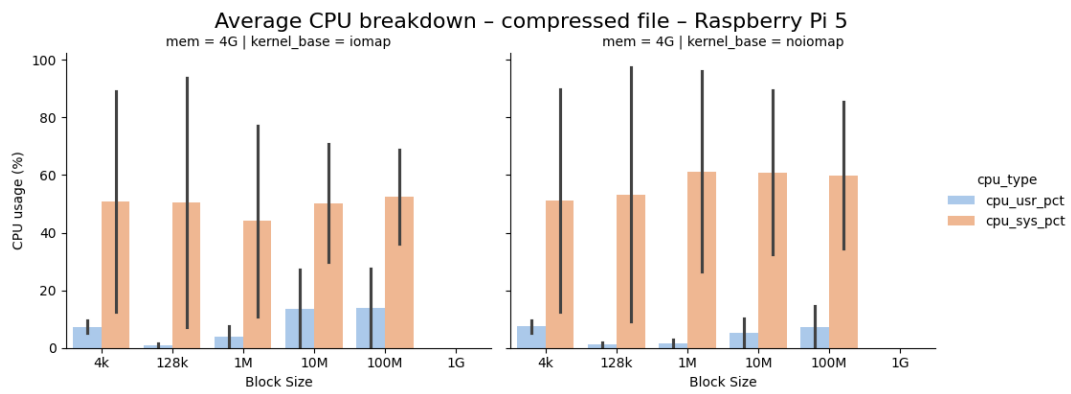


图 5-18 树莓派 5 平台压缩文件 CPU 利用率对比

结语

本文围绕 F2FS 在动态大页条件下的缓冲 I/O 适配问题展开研究。论文首先从移动端大文件访问、页缓存粒度、块层连续 I/O 能力和 F2FS 闪存友好机制出发，说明传统 4KB 页缓存粒度在现代连续 I/O 场景下面临的路径开销问题；随后结合 F2FS 的块映射、压缩簇、垃圾回收和私有状态管理机制，分析动态大页进入现有路径后引发的粒度失配、字段冲突和生命周期判定问题。

在设计与实现方面，本文将 F2FS 的块级映射结果转换为 iomap 字节区间描述，构建统一的 F2FS 扩展型动态大页状态对象，并针对普通读写、脏页写回、压缩文件读写和 GC 搬移路径分别完成适配。该方案在保持 F2FS 顺序追加写入、压缩簇语义和后台回收语义的前提下，使动态大页能够进入完整的缓冲 I/O 关键路径，并通过逐块状态位图和 pending 计数避免整动态大页写放大和提前完成判断。

实验结果表明，本文原型在 QEMU 和树莓派平台的大块顺序 I/O 场景中均能够提升吞吐能力，并降低部分小粒度页缓存管理和逐块 I/O 组织带来的 CPU 开销。后续工作可以继续围绕更复杂的混合随机负载、长期老化后的 GC 行为、不同页面大小配置以及 Android 真实应用负载展开验证，以进一步完善 F2FS 动态大页路径的稳定性和适用范围。

参考文献

- [1] Kim H, Agrawal N, Ungureanu C. Revisiting storage for smartphones [J]. ACM Transactions on Storage, 2012, 8(4): 1~25.
- [2] Li Z, Zhang G. StreamCache: Revisiting Page Cache for File Scanning on Fast Storage Devices [A]. Bagchi S, Zhang Y. Proceedings of the 2024 USENIX Annual Technical Conference (USENIX ATC '24) [C]. Santa Clara: USENIX Association, 2024. 1119~1134.
- [3] Lee C, Sim D, Hwang J-Y, Cho S. F2FS: A New File System for Flash Storage [A]. Schindler J, Zadok E. Proceedings of the 13th USENIX Conference on File and Storage Technologies (FAST '15) [C]. Santa Clara: USENIX Association, 2015. 273~286.
- [4] Gorman M. Understanding the Linux Virtual Memory Manager [M]. Upper Saddle River: Prentice Hall, 2004.
- [5] Rubini A, Corbet J, Kroah-Hartman G. Linux Device Drivers [M]. 3rd Edition. Sebastopol: O'Reilly Media, 2005.
- [6] Tanenbaum A S, Bos H. Modern Operating Systems [M]. 5th Edition. Boston: Pearson, 2023.
- [7] Navarro J, Iyer S, Druschel P, Cox A. Practical, transparent operating system support for superpages [A]. Draves R, van Renesse R. Proceedings of the 5th Symposium on Operating Systems Design and Implementation (OSDI '02) [C]. Boston: USENIX Association, 2002. 89~104.
- [8] Hildenbrand D, Schulz M, Amit N. Every Mapping Counts in Large Amounts: Folio Accounting [A]. Bagchi S, Zhang Y. Proceedings of the 2024 USENIX Annual Technical Conference (USENIX ATC '24) [C]. Santa Clara: USENIX Association, 2024. 1273~1282.
- [9] The Linux Kernel Organization. VFS iomap [EB/OL]. <https://docs.kernel.org/filesystems/iomap/>, [2026-04-20].
- [10] Google. 16 KB page size [EB/OL]. <https://source.android.com/docs/core/architecture/16kb-page-size/16kb>, [2026-04-20].
- [11] Denning P J. The working set model for program behavior [J]. Communications of the ACM, 1968, 11(5): 323~333.
- [12] Bovet D P, Cesati M. Understanding the Linux Kernel [M]. 3rd Edition. Sebastopol: O'Reilly Media, 2005.

- [13] Panwar A, Prasad A, Gopinath K. Making huge pages actually useful [A]. Gupta R, Amarasinghe S. Proceedings of the 23rd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '18) [C]. New York: ACM, 2018. 679~692.
- [14] Panwar A, Bansal S, Gopinath K. HawkEye: Efficient Fine-grained OS Support for Huge Pages [A]. Bahar I, Herlihy M. Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '19) [C]. New York: ACM, 2019. 347~360.
- [15] Mathur A, Cao M, Bhattacharya S, et al. The new ext4 filesystem: current status and future plans [A]. Hutton A. Proceedings of the Linux Symposium (OLS '07) [C]. Ottawa: OLS, 2007. 21~33.
- [16] Rodeh O, Bacik J, Mason C. Btrfs: The Linux B-tree Filesystem [J]. ACM Transactions on Storage, 2013, 9(3): 1~32.
- [17] Joshi K, Gupta A, González J, et al. I/O Passthru: Upstreaming a Flexible and Efficient I/O Path in Linux [A]. Ma X, Won Y. Proceedings of the 22nd USENIX Conference on File and Storage Technologies (FAST '24) [C]. Santa Clara: USENIX Association, 2024. 107~121.
- [18] Rosenblum M, Ousterhout J K. The design and implementation of a log-structured file system [J]. ACM Transactions on Computer Systems, 1992, 10(1): 26~52.
- [19] The Linux Kernel Organization. Flash-Friendly File System (F2FS) [EB/OL]. <https://docs.kernel.org/filesystems/f2fs.html>, [2026-04-20].
- [20] Zhang J, Shu J, Lu Y. ParaFS: A Log-Structured File System to Exploit the Internal Parallelism of Flash Devices [A]. Gulati A, Weatherspoon H. Proceedings of the 2016 USENIX Annual Technical Conference (USENIX ATC '16) [C]. Denver: USENIX Association, 2016. 87~100.

致谢

感谢我的导师在操作系统方面的悉心教学以及选题方面的指导,让我拥有了扎实的基本功以及明确的努力方向,并且感谢导师的持续鼓励,让我在面对晦涩复杂的 Linux 内核代码能鼓起勇气和信心,坚持到底。